Control Strategy: Nov27th Path Correction Code

Oct 28, 2026 · @Rasitha R

This document has two parts. **Part 1** describes the Nov27th code as it is today: how it works, its settings and its shortcomings. **Part 2** covers the improvements and future work: quick fixes, the look-ahead buffer, the per-step motion limiter, and how to transform the offset to the dispenser tip TCP.

Part 1 · Current design (Nov27th code)

Overview

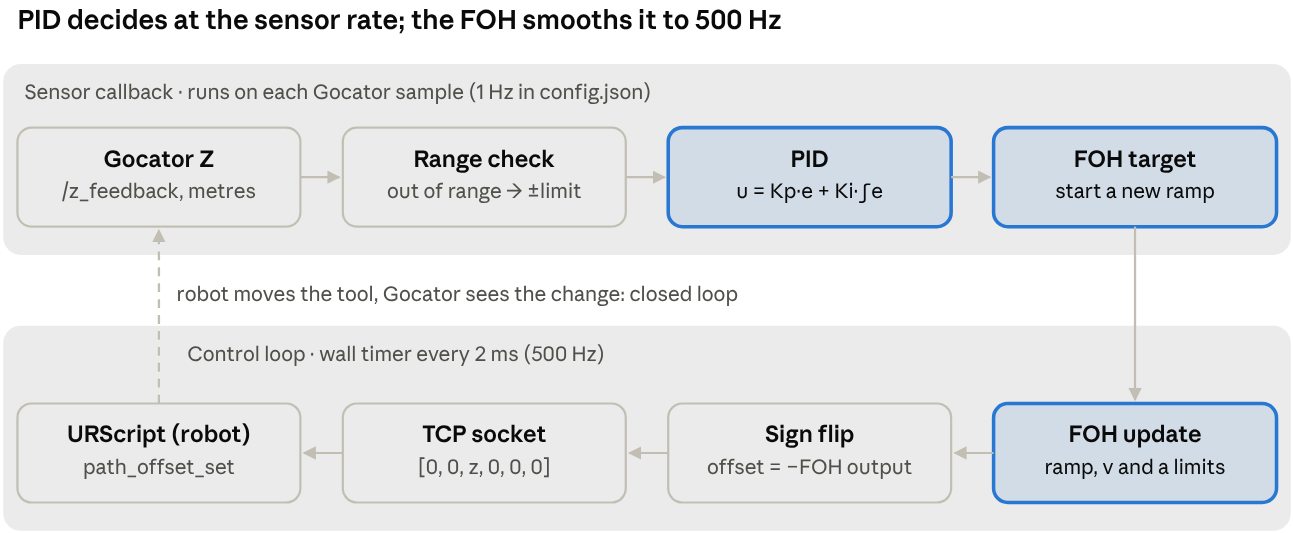
The Nov27th code is a single-axis (Z height) closed-loop correction: a PID turns the Gocator height error into an offset at the sensor rate, and a rate- and acceleration-limited first-order hold (FOH) stretches that offset into a smooth 500 Hz stream sent to the UR10e. It runs, but with the shipped gains the loop is unstable and several safety and timing issues need fixing before production use.

What runs today (`src/pathcorrection.cpp`, built as `dynamicontrol`):

- **Input:** Gocator Z value on ROS 2 topic `/z_feedback` (`std_msgs/Float32`, metres).
- **Control law:** PID on `error = setpoint − sensor_z`, with output clamping and conditional anti-windup.
- **Signal shaping:** FOH ramps from the previous command to the new PID target over one sensor period, with velocity and acceleration limits.
- **Output:** every 2 ms, `-FOH output` is sent over a TCP socket as `[0, 0, z, 0, 0, 0]`; a URScript program on the robot (not in the zip) applies it with `path_offset_set`.
- **Safety:** if the reading leaves the Gocator range (±12.5 mm), the command jumps to the offset limit (±30 mm).
- **Logging:** sensor, PID and FOH values every 2 ms, dumped to JSON or CSV on shutdown.

Older prototypes (`pathcorrection_thread.cpp`, `scripts/path_correction.py`) are in the zip but are not built and do not match the current headers.

Architecture and data flow



Nov27th data flow · two rates, one loop

The sensor callback and the 2 ms timer run in one single-threaded ROS 2 executor and share only the FOH target. The loop closes physically: the robot moves the tool, so the next Gocator reading already contains the applied offset.

Design strategy: why a PID and why an FOH

The design splits the problem into two rates: a slow "decide" loop that runs whenever a Gocator sample arrives, and a fast "actuate" loop that feeds the robot every 2 ms. The PID decides; the FOH bridges the gap between the two rates.

Why a PID

- **The sensor measures the result of the correction.** The Gocator rides on the flange, so after the robot moves, the next reading already includes that move. That is a closed loop, and a PID is the standard tool for driving a measured value (sensor Z) to a setpoint (−8 mm).
- **Integral action removes steady error.** If the part sits 2 mm lower over a long stretch, the integral keeps adding offset until the reading equals the setpoint again.
- **Proportional action reacts quickly** to a new error; derivative action was planned (filtered D with coefficient N) but is switched off (Kd = 0).

- **Output limits** keep the commanded offset inside ±30 mm (config), protecting the part and the robot.

Why an FOH

- **Rate mismatch.** The Gocator data arrives at 1–30 Hz (config: 1 Hz or 5 Hz); `path_offset_set` is applied at 500 Hz. Sending a new PID value only when it changes would produce a staircase: a step every sensor period.
- **Steps cause protective stops.** If the offset changes too much from one step to the next, the UR controller stops the robot with error **C219 Path Offset**. It has three variants: C219A1 (the joint speed limit would be exceeded), C219A2 (the tool speed limit would be exceeded) and C219A3 (the momentum limit would be exceeded). UR's suggested fix is a smaller change per step or more filtering with `path_offset_set_alpha_filter()`, so the command must be smooth.
- **First-order hold = straight-line interpolation.** Instead of jumping to the new PID value, the FOH ramps from the previous value to the new one over one sensor period, so the robot sees a continuous, piecewise-linear command.
- **Extra protection.** On top of the ramp, the FOH caps the offset velocity (`max_rate_limit`) and acceleration (`max_accel_limit`), so even a large PID step or a fail-safe command is applied gently.

Why not a zero-order hold (ZOH)

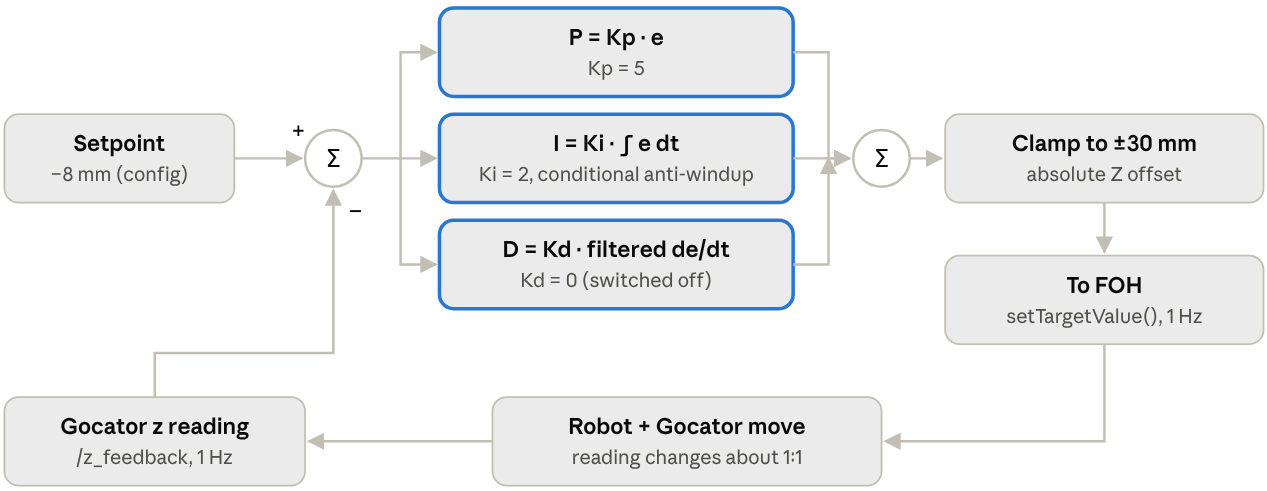
A `ZeroOrderHold` class exists and CMake has a `USE_ZOH` option, but the ZOH source file (`pathcorrection_zoh.cpp`) is not in the zip. A ZOH holds each value until the next sample, giving steps at every sample, which is exactly what the FOH avoids.

The price of this design

The FOH reaches each new target only at the end of the sensor period, so it adds about one sensor period of delay (1 s at the shipped 1 Hz). In a closed loop, extra delay plus high gain leads to oscillation; see Shortcomings.

PID design in detail

`include/utils/PIDController.hpp` implements a positional PID with automatic time step, a filtered derivative and conditional anti-windup. It runs once per Gocator sample, inside `sensorCallback`.



Runs once per Gocator message. Gain per sample ≈ Kp + Ki·T ≈ 7 (needs to be below 1), so the loop oscillates.

Current PID loop, as in PIDController.hpp and config.json

The PID closes its loop through the robot. Every offset it sends moves the Gocator, which changes the next reading by about the same amount. With Kp = 5 and Ki = 2 at 1 Hz, that loop multiplies each error by about 7, which is why it oscillates.

Error and terms

```
error  = setpoint − sensor_z            (metres; setpoint = −0.008 in
config.json)
P      = Kp · error                     (Kp = 5)
I      = Ki · Σ(error · dt)             (Ki = 2)
D      = Kd · filtered d(error)/dt       (Kd = 0, so off)
output = clamp(P + I + D, minOffset, maxOffset)   (±0.03 m)
```

The output is an **absolute offset** (not an increment): each sample replaces the previous target.

Time step

- `useChrono = true`: dt is measured with `rclcpp::Clock()` between consecutive calls, so it equals the real gap between Gocator messages.
- dt ⩽ 0 is replaced by 1e-6 s to avoid division by zero.
- The clock starts in the constructor, so the first dt includes the start-up time (see Shortcomings).

Derivative filter

First-order low-pass on the raw derivative:

```
alpha              = dt / (dt + 1/N)          (N = derivative_filter_N = 10)
derivative_filtered = alpha · raw + (1 − alpha) · previous
```

With N = 10 the filter time constant is 0.1 s. It has no effect today because Kd = 0.

Saturation and anti-windup

- Output is clamped to [minOffset, maxOffset].
- If the output saturates **and** the error pushes further into saturation, the integral update of this step is undone (`integral −= error · dt`). This is conditional integration: the integral stops growing while the output is pinned.

Loop gain

Because the Gocator moves with the robot, a 1 mm offset change changes the reading by about 1 mm. Per sample, the PID therefore applies roughly **Kp + Ki·T** times the error:

Config	Kp	Ki	Sensor period T	Effective gain per sample	Expected behaviour
config.json	5	2	1.0 s	≈ 7	Strong oscillation (simulated ±5 mm on a 2 mm step)
config_Sensor-5Hz.json	1	1	0.2 s	≈ 1.2	Mild oscillation (simulated ±0.7 mm)
Stable range	0	any	T	0.3–0.7	Converges without overshoot

The simulations are from an earlier review of this code (static 1:1 plant plus the FOH, no robot dynamics); they show the trend, not exact amplitudes.

Call sequence and timing

1. A Gocator message arrives on `/z_feedback` ; `sensorCallback` runs.
2. If the reading is inside ±12.5 mm, `pid_->update(setpoint_z_, sensor_z)` is called once; otherwise the PID is skipped (fail-safe).
3. Inside `update()` : measure dt since the last call → compute the error → P, I, filtered D → sum → clamp → anti-windup correction → store `prevError` and `last_output_` .

4. The result is stored in `latest_pid_output_` (for logging) and handed to the FOH with `setTargetValue()`.
5. The PID is idle until the next Gocator message; the 500 Hz loop never calls it.

So the PID runs at the Gocator rate, its time step is the real gap between messages, and a missing message simply means no update (the integral does not grow during gaps).

Worked example (config.json values)

Setpoint = −8.0 mm, Gocator reads −6.0 mm, one second since the last sample, integral starts at 0:

Step	Calculation	Result
Error	−0.008 − (−0.006)	−0.002 m (−2 mm)
P term	5 × (−0.002)	−0.010 m
Integral	0 + (−0.002 × 1.0)	−0.002 m·s
I term	2 × (−0.002)	−0.004 m
D term	Kd = 0	0
Output	−0.010 + (−0.004), inside ±0.03	−0.014 m (−14 mm)
Sent to robot	× −1 after the FOH	+14 mm

A 2 mm height error produces a 14 mm move. Because the Gocator moves with the robot, the next reading swings about 12 mm the other way, which is why the loop oscillates (Shortcomings #1).

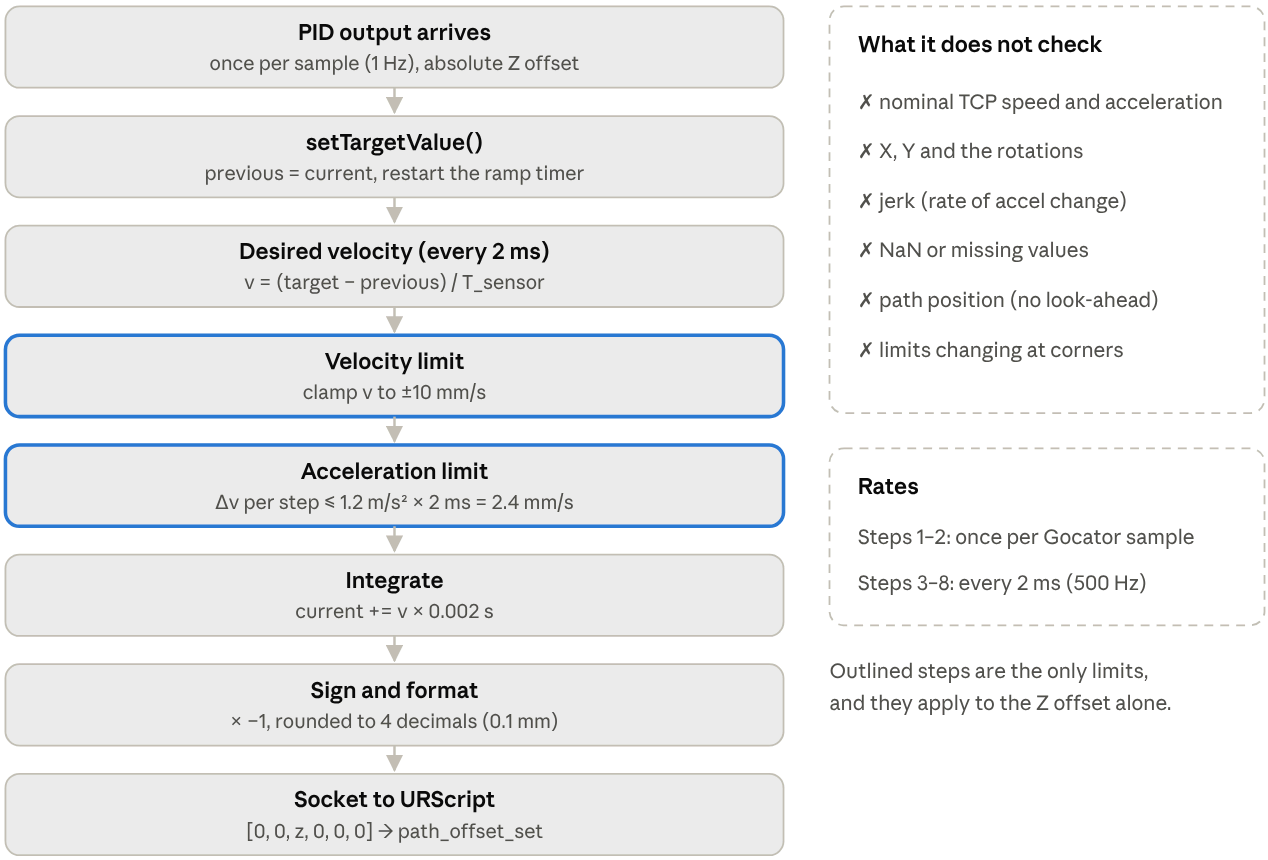
Strengths of the current PID design

- **Clear, standard structure:** P, I and D are separate, easy to read and to tune; gains come from config, not code.
- **Real time step:** dt is measured, so the integral stays correct even if the Gocator rate drifts or a sample is late.
- **Output saturation:** the command can never exceed ±maxOffset, which bounds the worst case.
- **Anti-windup built in:** conditional integration stops the integral from growing while saturated, so recovery after a long saturation is quick.

- **Derivative already filtered:** if D is ever switched on, sensor noise is not amplified unfiltered (N = 10).
- **Safe division:** dt ⩽ 0 is guarded.
- **Inspectable:** `getLastOutput()` and the logging make every PID output visible for tuning.
- **Reusable:** header-only class with no ROS topics inside, so it can be unit-tested on its own.

FOH design in detail

`include/utils/FirstOrderHold.hpp` is not a pure first-order hold: it is a linear ramp with a velocity limit and an acceleration limit, effectively a small trajectory generator. `setTargetValue()` is called by the sensor callback; `update()` is called by the 500 Hz control loop.



Current FOH, as in FirstOrderHold.hpp, pathcorrection.cpp and socket_utils.cpp

Steps 1–2 run once per Gocator sample; steps 3–8 run every 2 ms. The two limits act on the Z offset only, with fixed values.

When a new target arrives (`setTargetValue`)

```
previous_value = current_value     (start the new ramp from where the output
is now)
target_value   = new PID output
previous_time  = now
```

Every 2 ms (`update`)

1. **Desired velocity**
   - Within one sensor period of the new target: `v_des = (target − previous) / T_sensor` (the straight-line FOH slope).
   - After the sensor period has passed (late or missing sample): `v_des = (target − current) / T_act`, i.e. try to close the remaining gap in one 2 ms step.
2. **Velocity limit:** `v_des = clamp(v_des, ±max_rate_limit)` (0.01 m/s = 10 mm/s in config).
3. **Acceleration limit:** the velocity may change by at most `max_accel_limit · T_act` per step (1.2 m/s² × 0.002 s = 2.4 mm/s per step).
4. **Integrate:** `current += v · T_act`; return `current`.

What this means in practice

Situation	Behaviour
Small change (e.g. 1 mm at 1 Hz)	Straight ramp over 1 s at 1 mm/s; target reached at the end of the period
Large change (e.g. 30 mm fail-safe)	Capped at 10 mm/s, so it takes about 3 s; the ramp outlasts the sensor period and continues in "catch-up" mode
Ramp start/end	Velocity ramps up and down at 1.2 m/s² (about 8 ms to reach 10 mm/s), so no velocity steps
Missing samples	After one period, it keeps moving toward the last target and then holds it
Near the target in catch-up mode	Desired velocity is error/0.002 s, limited by the acceleration cap; can overshoot by a few hundredths of a mm and settle

Units

All values are metres and m/s. Comments in the file say "mm" in places (`new_target is in mm`); the code itself works in metres, consistent with config.json.

Sign

`controlLoop()` multiplies the FOH output by −1 before sending. The sign convention between Gocator Z, the PID error and the robot offset frame is therefore fixed in code; it must match the robot script's frame (base or TCP), which is not in the zip.

Call sequence and timing

1. **Sensor callback** (Gocator rate): `foh_->setTargetValue(pid_output)` stores the new target, freezes the start point (`previous_value = current_value`) and restarts the ramp clock.
2. **Control loop** (every 2 ms): `foh_->update()` computes the desired velocity, applies the velocity and acceleration limits, integrates one step and returns the new offset.
3. The loop multiplies the result by −1, logs it and sends it to the robot.

The FOH holds its own state (`current_value_`, `current_velocity_`), so the robot command is continuous in position and velocity even when the PID target jumps.

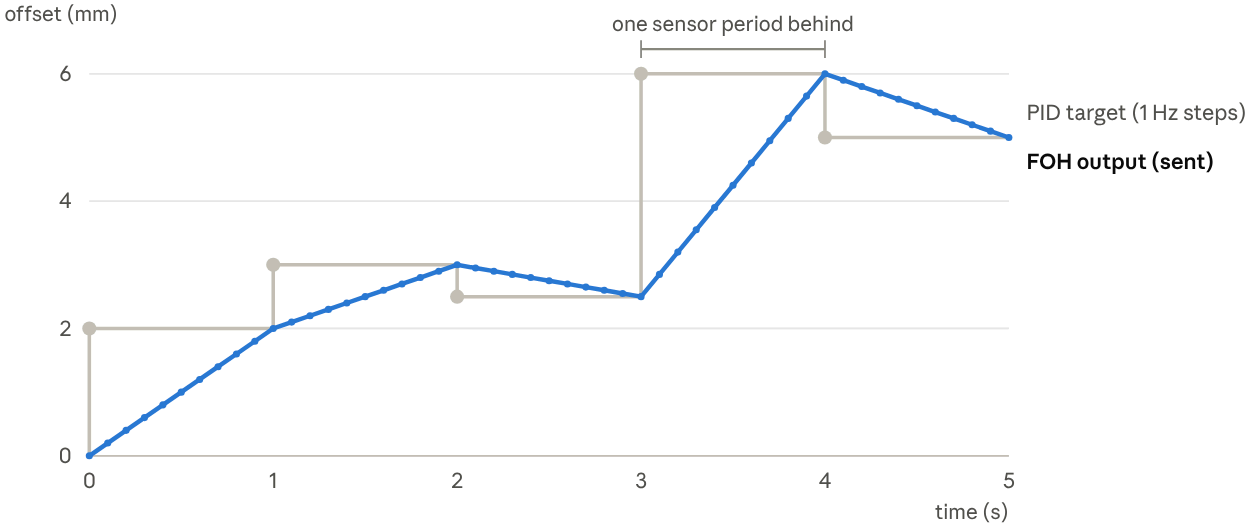
Worked examples (config.json: 1 Hz, 10 mm/s, 1.2 m/s²)

Case	What the FOH does	Time to reach target
PID target 0 → 2 mm	Ramp at 2 mm/s (below the 10 mm/s cap); velocity reaches 2 mm/s within one 2 ms step	1.0 s (end of the sensor period)
PID target 2.5 → 6 mm	Ramp at 3.5 mm/s	1.0 s
Fail-safe 0 → 30 mm	Wants 30 mm/s, capped at 10 mm/s; continues in catch-up mode after the first second	about 3.0 s
No new sample	Finishes the current ramp, then holds	—

The chart below shows the FOH output for a sequence of 1 Hz PID targets (simulated with the FOH code's logic and the config.json limits).

The FOH reaches each PID target one sensor period later, as a straight ramp

Offset in mm vs time in s · config.json: 1 Hz sensor, 10 mm/s and 1.2 m/s² limits

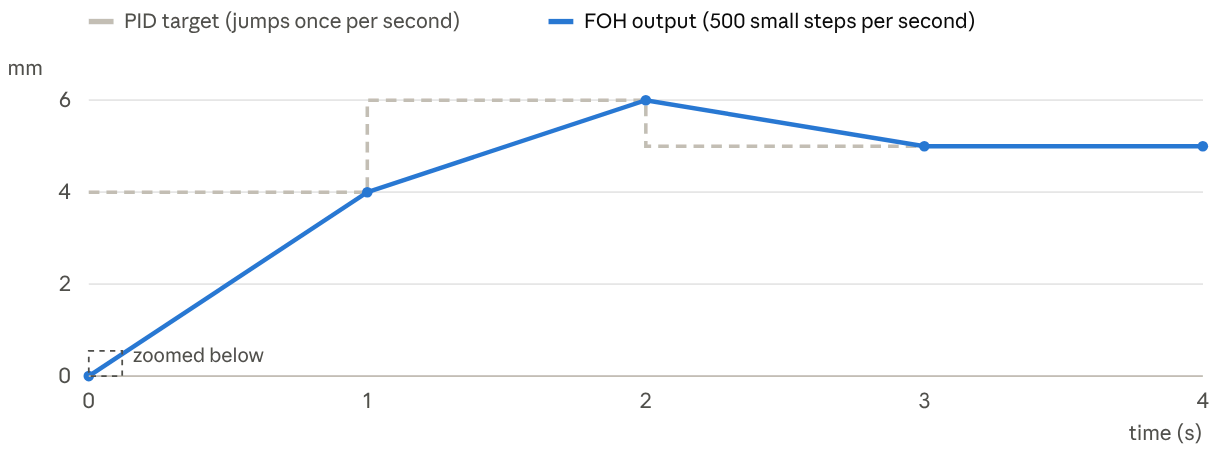


Simulated with the FOH logic from FirstOrderHold.hpp and config.json limits · 5 PID targets at 1 Hz

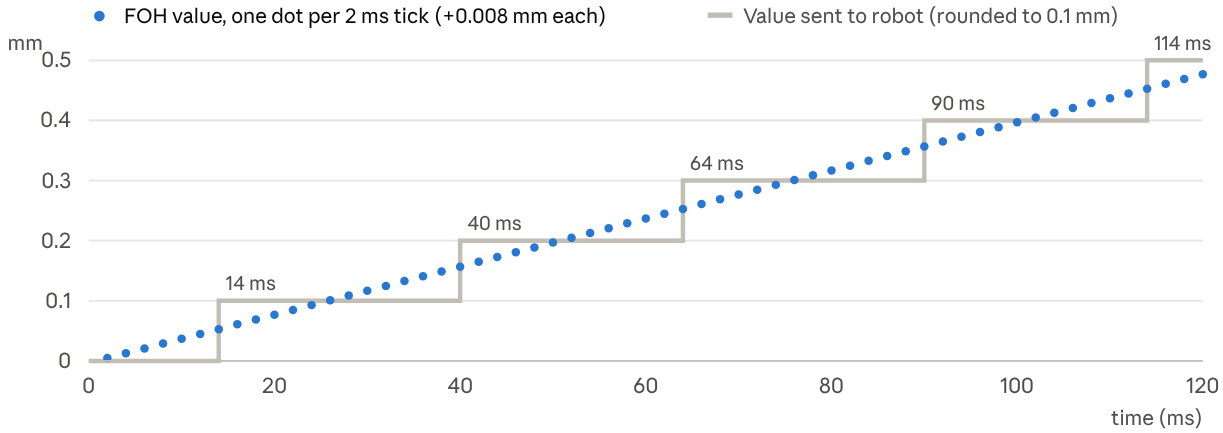
The ramp keeps the robot command smooth, but each PID decision only fully reaches the robot one second later, and inside a closed loop that delay feeds the oscillation described in Shortcomings #1.

Step by step: how the FOH sends the correction

1 · Whole view: a new PID target every 1 s, the FOH ramps between them



2 · Zoom on the first 120 ms: one FOH step every 2 ms, but only 0.1 mm steps are sent



Simulated tick by tick with the FirstOrderHold.hpp logic and config.json limits · example PID targets 4, 6, 5 mm · values before the × −1 sign flip

How it smooths. The PID gives a new target only once per second (top chart, dashed). If that jump were sent directly, the robot would jump 4 mm in one go. Instead, every 2 ms the FOH moves a small step towards the target, so the 4 mm is spread over 500 steps of 0.008 mm (top chart, solid line). Before each step it applies the two limits: speed at most 10 mm/s, and speed change at most 2.4 mm/s per tick.

What actually reaches the robot. The socket rounds the value to 4 decimals in metres, which is 0.1 mm. So the robot sees no change for the first 12 ms, then a 0.1 mm jump about every 25 ms (bottom chart, grey). Each of those jumps happens inside one 2 ms tick, which is about 50 mm/s for that instant: much faster than the 10 mm/s the FOH intended (Shortcomings: rounding). Sending 6 decimals fixes this.

Tick	Time	What happens	FOH speed	FOH value	Sent
1	2 ms	Target 4 mm arrives; wanted speed (4 − 0) / 1 s = 4 mm/s, but speed may only grow 2.4 mm/s per tick	2.4 mm/s	0.0048 mm	0.0 mm
2	4 ms	Speed reaches the wanted 4 mm/s (below the 10 mm/s cap)	4.0 mm/s	0.0128 mm	0.0 mm
3–6	6–12 ms	Each tick adds 4 mm/s × 2 ms = 0.008 mm	4.0 mm/s	0.0208 → 0.0448 mm	0.0 mm
7	14 ms	Value passes 0.05 mm, so the rounded value changes	4.0 mm/s	0.0528 mm	0.1 mm
20	40 ms	Next rounding step	4.0 mm/s	0.1568 mm	0.2 mm
500	1.000 s	End of the sensor period; target almost reached	4.0 mm/s	3.9968 mm	4.0 mm
501	1.002 s	New target 6 mm; wanted speed (6 − 4) / 1 s = 2 mm/s; speed drops 4 → 2 within the 2.4 limit	2.0 mm/s	4.0008 mm	4.0 mm
1001	2.002 s	New target 5 mm; wanted −1 mm/s; speed can only drop 2.4 per tick	−0.4 mm/s	5.9992 mm	6.0 mm
1002	2.004 s	Speed reaches −1 mm/s and ramps down to 5 mm by 3 s	−1.0 mm/s	5.9972 mm	6.0 mm

The speed limits are only reached at the very start of each ramp, for one tick. The rest of the time the ramp simply follows the straight line from the old target to the new one, over one sensor period. Since the next target arrives just as the ramp ends, the robot is always one sensor period (1 s) behind the PID.

Strengths of the current FOH design

- **Removes steps:** a 1–30 Hz decision becomes a continuous 500 Hz command, avoiding the staircase a zero-order hold would send.
- **Continuous velocity:** the acceleration limit means even the ramp corners are rounded, which is what the UR path-offset checks (C219) want.
- **Hard speed cap:** no matter what the PID or the fail-safe asks for, the offset cannot move faster than `max_rate_limit`. This is the main reason a bad target is not dangerous instantly.
- **Handles late samples:** the catch-up mode keeps moving toward the last target and then holds, instead of extrapolating into nowhere.

- **Starts from where it is:** each new ramp starts at the current output, so overlapping or early targets never cause a jump.
- **Configurable:** sensor period, velocity and acceleration limits come from config.
- **Reusable as a final shaping stage:** in the target architecture (Kalman filter + look-ahead + geometric offset), this same limiter can stay as the last step before the robot.

What the FOH limits: the Z offset (and so the TCP's Z motion), not the total TCP motion

The FOH limits the **offset** (the correction), one 2 ms step at a time. The nominal TP path is taught at a **fixed Z height**: straight segments, plus curves in the travel plane where the part curves. The nominal path does not handle the part's ups and downs: its Z value stays constant along the whole path. Any change in the TCP's Z therefore comes **only from the offset**, and the FOH limits are also the **TCP's Z distance, velocity and acceleration** per robot update. What the FOH does **not** limit is the **total** TCP motion: the travel speed along the path combined with the Z correction ($\sqrt{(v_travel^2 + v_z^2)}$), and on curves the turning acceleration v^2/R of the nominal path combined with the Z offset acceleration.

Per robot update (2 ms)	Limited by the FOH?	Value with config.json
TCP Z distance per step (all from the offset)	Yes, through the speed cap	⩽ 0.02 mm
TCP Z velocity (all from the offset)	Yes (`max_rate_limit`)	⩽ 10 mm/s
TCP Z acceleration (all from the offset)	Yes (`max_accel_limit`)	speed changes ⩽ 2.4 mm/s per step (1.2 m/s²)
Jerk (change of acceleration)	No	can switch from 0 to 1.2 m/s² in one step
Total offset (±30 mm)	No; only the PID output is clamped	FOH output is not clamped
Total TCP speed and acceleration (travel + curve turning + Z offset)	No	the FOH does not know the nominal motion
Joint speeds and accelerations	No	only the robot's safety system checks these

The comment in `FirstOrderHold.hpp` calls `max_rate_limit` the "max TCP velocity"; the code applies it to the Z offset only.

The FOH limit is also a TCP Z limit only when all of these hold:

1. **The nominal path stays at its fixed Z height.** This is true for the current TP program: straight segments and curves in the travel plane only, with no up or down moves. If Z moves are ever added (ramps, approach or retract moves during dispensing), the TCP's Z velocity becomes nominal Z velocity + offset Z velocity, and only the offset part would be limited.
2. **The offset frame matches.** With base Z (type 1) and a vertical tool, base Z = tool Z. With a tilted tool and tool frame (type 2), the offset is along the tilted tool axis.
3. **The robot receives what the FOH computes.** Two parts of the current code break this:
   - **Rounding:** 4 decimals in metres means the robot sees 0.1 mm jumps every few steps instead of 0.02 mm each step, about 50 mm/s for one cycle (5× the limit).
   - **Timing:** the FOH assumes exactly 2 ms per call. Wall-timer jitter, the per-cycle `std::cout`, and queued TCP messages can put two or more steps into one robot cycle.
4. **The limit values suit the robot.** 10 mm/s and 1.2 m/s² were chosen, not derived from the UR10e or from C219 tests. They are gentle, but should be confirmed on the robot.

Two further gaps in the limit chain:

- **NaN passes every check.** A NaN reading passes the range check (both comparisons are false), skips the PID clamp (same reason), makes the integral NaN for ever, and is sent to the robot as "nan".
- **No final guard before sending.** Nothing clamps the value that leaves the PC; the FOH can overshoot a target by about $v^2/2a \approx 0.04$ mm, and a missing config key silently changes the limit to the code default.

Other conditions and criteria handled

Condition	How the code handles it	Where
Sensor reading out of range (below −12.5 mm or above +12.5 mm)	Skips the PID and commands the full offset limit: below range → `minOffset`, above range → `maxOffset` (then ×−1 on output). The PID state is left untouched	`sensorCallback`
Offset limit	PID output clamped to ± `maxOffset` (±30 mm)	`PIDController::update`
Integral windup	Conditional integration while saturated	`PIDController::update`
Smooth robot motion	FOH ramp + velocity and acceleration limits	`FirstOrderHold::update`
Late or missing Gocator samples	FOH finishes the ramp, then holds the last target; the PID is not called	`FirstOrderHold::update`
Rate mismatch (sensor vs robot)	Separate callback (sensor rate) and 2 ms wall timer (500 Hz)	constructor
Thread safety	Shared values in `std::atomic`; log vector behind a mutex. With the default single-threaded executor, callbacks never run at the same time, so this is a safeguard	class members
Configuration	All gains, limits and rates read from `config/config.json` at start-up, with built-in defaults	constructor
Data logging	Sensor Z, PID output and FOH output stored every 2 ms; written to JSON or CSV in the destructor	`controlLoop`, `dump_data`

Condition	How the code handles it	Where
Robot connection	The PC opens a TCP server (port 50003) and blocks until the robot connects	`initServer`, `acceptConnection`

Not handled: look-ahead distance between Gocator and nozzle, timestamps, robot pose feedback, wrong-feature or cutout readings, more than one axis, reconnection, and a watchdog on the PC side.

Configuration parameters

The node reads a hard-coded path (`/home/ws/Nov_27th/.../config/config.json`). Defaults apply when a key is missing.

Key	config.json	config_Sensor-5Hz.json	Code default	Effect
setpoint	−0.008 m	0.0 m	0.05 m	Gocator Z the loop drives to
Kp	5.0	1.0	1.0	Proportional gain
Ki	2.0	1.0	0.5	Integral gain (1/s)
Kd	0.0	0.0	0.0	Derivative gain (off)
derivative_filter_N	10	10	10	D-filter bandwidth (1/s)
minOffset / maxOffset	±0.03 m	±0.02 m	±0.02 m	PID clamp and fail-safe command
actuating_rate	500 Hz	500 Hz	500 Hz	Control-loop and send rate
sensor_rate	1 Hz	5 Hz	30 Hz	FOH ramp period (must match the real Gocator rate)
max_rate_limit	0.01 m/s	0.01 m/s	1.5 m/s	FOH velocity cap
max_accel_limit	1.2 m/s²	1.2 m/s²	25 m/s²	FOH acceleration cap

Key	config.json	config_Sensor-5Hz.json	Code default	Effect
sensor_range_min / max	±0.0125 m	±0.0125 m	±12.5 (no unit)	Out-of-range fail-safe threshold
ROBOT_IP / ROBOT_PORT	169.254.0.1 / 50003	same	same	Address the PC server binds to (the robot connects to it)
logging_mode	json	csv	csv	Dump format

Two defaults are in the wrong unit: the sensor range default (12.5) is millimetres while everything else is metres, and the default rate and acceleration limits (1.5 m/s, 25 m/s²) are far above what the config uses.

Files reviewed

File	Role	Built?
`src/pathcorrection.cpp`	Main ROS 2 node: sensor callback, PID, FOH, 500 Hz loop, logging	Yes (`dynamicontrol`)
`include/utils/PIDController.hpp`	PID with auto dt, filtered D, anti-windup	Yes
`include/utils/FirstOrderHold.hpp`	Ramp with velocity and acceleration limits	Yes
`include/utils/ZeroOrderHold.hpp`	Sample-and-hold (included, unused)	Header only
`src/utils/socket_utils.cpp`, `include/utils/socket_utils.hpp`	TCP server/client, offset message formatting	Yes
`config/config.json`, `config/config_Sensor-5Hz.json`	Gains, limits, rates	Read at start-up
`CMakeLists.txt`, `src/CMakeLists.txt`	Build; `USE_ZOH` option points to a missing file	—

File	Role	Built?
`src/pathcorrection_thread.cpp`	Older 30 Hz prototype, client to port 30002	No (does not match current headers)
`scripts/path_correction.py` + `scripts/utils/*.py`	Python prototype with PID, FOH, ZOH, socket	No
`scripts/gocator_publisher_dummy.py`	Test publisher (0–10 "mm" pulse on `/sensor/z_feedback`, Float64)	Does not match the node's topic, type or unit
`scripts/plot_control_data.py`	Plots the CSV dump	—

Not in the zip: the URScript program running on the robot, the Gocator/Vision node that publishes `/z_feedback`, and any test data.

Shortcomings

This section lists what is wrong or risky in the Nov27th code. Each item first appears in the summary table, then is explained in three parts: **what the code does** (with the file), **why it is a problem** (with a concrete example) and **how to fix it**. Severity: **High** = can damage the part or nozzle, or makes the result wrong; **Medium** = degrades quality or hides failures; **Low** = maintenance or clarity.

Summary

#	Shortcoming (plain words)	Severity	Where
1	The PID reacts about 7 times too strongly, so the correction swings up and down	High	`config.json`, `PIDController.hpp`
2	One bad Gocator reading sends the nozzle towards the full ±30 mm	High	`sensorCallback`
3	The very first correction after start-up is a big jump	High	PID constructor, node constructor

#	Shortcoming (plain words)	Severity	Where
4	The reading taken L mm ahead is applied at the nozzle straight away	Medium	design
5	Only one correction per second, applied over the following second	Medium	`config.json` (`sensor_rate`)
6	The value sent to the robot only changes in 0.1 mm jumps	Medium	`socket_utils.cpp` (`sendOffset`)
7	The robot-side script is missing and the message format may not parse	Medium	`sendOffset` , URScript (not in zip)
8	Old offsets can pile up in the network connection	Medium	`controlLoop` , `sendOffset`
9	Readings have no time, and nobody checks what the robot really did	High	design
10	Any in-range reading is trusted, even from a wrong feature	High	`sensorCallback`
11	Lost connections and missing sensor data are not detected	High	constructor, `sendOffset`
12	Units are mixed (m and mm), and missing settings fall back to wrong values	Medium	constructor, comments
13	Logged data is kept in memory and lost if the program crashes	Low	`controlLoop` , `dump_data`
14	File paths are fixed to one computer	Low	constructor, dump functions

#	Shortcoming (plain words)	Severity	Where
15	Old, unused and mismatched files in the repository	Low	repo
16	The Python prototype would stop the robot program	Low	`scripts/utils/ur_socket.py`
17	A NaN reading breaks the controller until restart	High	`sensorCallback`, `PIDController::update`
18	FOH limits are offset limits; total TCP motion is not checked	Low today, High for 6D	`FirstOrderHold.hpp`
19	Acceleration switches on in a single step (no jerk limit)	Low	`FirstOrderHold::update`
20	Nothing checks the value just before it is sent	Medium	`controlLoop`
21	The FOH assumes every call is exactly 2 ms apart	Low	`FirstOrderHold::update`

Each shortcoming explained

1 · The PID reacts about 7 times too strongly

What the code does. `config.json` sets Kp = 5 and Ki = 2, and the PID runs once per Gocator message (1 Hz). Per sample it moves the offset by roughly (Kp + Ki × 1 s) × error ≈ 7 × error.

Why it is a problem. The Gocator is on the same flange as the nozzle, so when the robot moves the offset by 1 mm, the next Gocator reading also changes by about 1 mm. For such a loop to settle, each correction must be *smaller* than the error it is correcting (gain below 1). Example from the worked example above: the nozzle is 2 mm too high, the PID asks for a 14 mm move; one second later it is now 12 mm too low, so the PID asks for a large move back, and so on. The simulation shows a steady ±5 mm swing on a 2 mm step; the 5 Hz config (Kp = Ki = 1) still swings about ±0.7 mm. The 1 s FOH ramp adds delay and makes it worse.

Fix. Incremental, integral-only update `offset += k · error` with k = 0.3–0.7 per sample (Part 2, PID improvements).

2 · One bad reading sends the nozzle towards ±30 mm

What the code does. In `sensorCallback`, a reading below −12.5 mm sets the target to `minOffset` (−30 mm), a reading above +12.5 mm sets it to `maxOffset` (+30 mm). After the `× −1` in `controlLoop`, the robot receives +30 or −30 mm.

Why it is a problem. An out-of-range reading usually means the Gocator lost the edge (end of the part, a cutout, a reflection, sealant on the edge), not that the part really moved by 12.5 mm. Yet a single such reading makes the FOH move towards 30 mm at its full 10 mm/s, so the nozzle travels about 10 mm before the next reading arrives one second later. Towards the part this can hit it; away from it the bead is lost. The direction is also hard to predict: the log says "MAX UP" for a low reading, but the code (where the two lines were swapped) commands `minOffset`, which the `× −1` flips again.

Fix. On an invalid reading, **hold** the last good offset (or fade slowly to 0), count consecutive bad readings, and after N of them stop dispensing and raise an alarm.

3 · The first correction after start-up is a big jump

What the code does. The PID stores its start time in its constructor (`pathcorrection.cpp`, line 76). Then the node waits in `acceptConnection()` (line 90) until the robot program connects, and the first Gocator message comes even later. The first `update()` therefore uses dt = the whole waiting time.

Why it is a problem. Example: 60 s wait, error −2 mm. The integral becomes −0.002 × 60 = −0.12 m·s, so the I term is 2 × (−0.12) = −0.24 m, which saturates at −30 mm. The anti-windup removes the integral again, but this first output is already ±30 mm, and the FOH starts moving the nozzle about 10 mm before the second reading arrives. That happens right at the start of the bead.

Fix. Start the PID clock at the first Gocator sample, and clamp dt to a sensible range (for example 0.5–2 × the sensor period).

4 · The reading taken L ahead is applied at the nozzle straight away

What the code does. The Gocator line is L mm ahead of the nozzle. Each reading is turned into a correction and applied immediately.

Why it is a problem. The correction is meant for the place the Gocator is looking at, but it is applied where the nozzle is now. On a flat part it doesn't matter. On a slope or a step it does: with L = 60 mm and a 2% slope (about 1.2°), the nozzle is corrected for a height 1.2 mm different from where it really is. At a step, the nozzle starts moving L mm too early. The 1 s FOH ramp shifts the correction back by v × 1 s, which only matches L by coincidence, at one travel speed (v = 60 mm/s for L = 60 mm).

Fix. Store each reading by its position along the path and use it when the nozzle reaches that position (Part 2, look-ahead buffer).

5 · Only one correction per second

What the code does. `config.json` sets `sensor_rate` = 1 Hz. The PID decides once per second and the FOH spreads each decision over the next second.

Why it is a problem. At 50 mm/s the robot travels 50 mm between two decisions. Anything shorter than that (a bump, a joint, a short ramp) is missed or corrected late. In the worst case a change waits up to 1 s for the next reading plus 1 s of ramp: 2 s, or 100 mm of bead.

Fix. Run the Gocator and the controller at 50–100 Hz or more (the sensor itself can do much faster), with the gain adjusted.

6 · The value sent to the robot only changes in 0.1 mm jumps

What the code does. `sendOffset` in `socket_utils.cpp` writes the offset with `std::setprecision(4)`. The value is in metres, so 4 decimals is 0.0001 m = **0.1 mm**. Anything finer is rounded away.

Why it is a problem. The FOH carefully computes small steps: at 4 mm/s that is 0.008 mm every 2 ms. After rounding, the robot sees no change for about 12 ms, then a whole 0.1 mm in a single 2 ms cycle (see "Step by step: how the FOH sends the correction"). A 0.1 mm move in 2 ms is **50 mm/s**, 5 times the 10 mm/s the FOH limits to, and the acceleration of that jump is far above 1.2 m/s². So the smoothing done by the FOH is partly undone at the very last step, just before the robot. Jerky offsets are exactly what UR warns cause C219 protective stops, and they can show as small ripples in the bead. UR's `path_offset_set_alpha_filter` on the robot side may soften it, but the script isn't in the zip, so we can't tell.

Fix. Send 6 decimals (1 µm), or send the value in mm, or use RTDE registers (item 9), which carry full double precision.

7 · The robot-side script is missing and the message may not parse

What the code does. The PC sends the text `[0, 0, z, 0, 0, 0]` every 2 ms. The URScript that reads it (with `socket_read_ascii_float`) and calls `path_offset_set` is not in the zip.

Why it is a problem. UR documents `socket_read_ascii_float` as expecting numbers in round brackets, like `(0, 0, 0.001, 0, 0, 0)`; square brackets may or may not be accepted. If parsing fails, the robot reads nothing and keeps the last offset, silently. Because the script is missing, we also can't check which frame it uses (base or tool), its alpha filter, its max offset, or what it does on a timeout. Half of the control loop cannot be reviewed.

Fix. Put the URScript in the repository, use round brackets, check that `socket_read_ascii_float` returns 6 values, and document the frame.

8 · Old offsets can pile up in the connection

What the code does. The PC sends one message every 2 ms from a ROS wall timer. The robot reads them in its own loop. Nothing keeps the two in step. Each cycle also prints two console lines (`sockfd = …`, `Sent: …`) plus ROS log lines.

Why it is a problem. Two clocks are never exactly equal. If the PC sends even slightly faster than the robot reads, or sends a burst after a hiccup, messages queue in the TCP buffer and the robot reads the oldest first. With 100 messages queued, the robot is applying an offset that is 200 ms old, and the delay keeps growing. Printing to the console every 2 ms takes time and makes the timer late, then catch up with two sends close together.

Fix. Have the robot read only the newest value, or better, send the offset through RTDE input registers, which always hold the latest value and are synchronised with the 500 Hz robot cycle (item 9). Remove the per-cycle prints.

9 · Readings have no time, and nobody checks what the robot really did

What the code does. The Gocator value arrives on `/z_feedback` as a plain `std_msgs/Float32`: a number with no time stamp. The PC never reads the robot's state, and the robot never reports which offset it applied.

Why it is a problem. Without a time stamp, we don't know *where the robot was* when the profile was taken, so the edge can't be placed in the base frame, the look-ahead buffer can't be built, and the Gocator's processing delay can't be compensated. Without read-

back, the PC can't tell whether the robot actually applied the offset (the message may not parse, item 7; it may be queued, item 8; the robot may be in a protective stop). The log shows what the PC wanted, not what happened.

Fix. See "How to add timestamps and robot feedback" below.

10 · Any in-range reading is trusted

What the code does. Any value between −12.5 and +12.5 mm goes straight into the PID.

Why it is a problem. The Gocator also sees cutouts, holes, a second edge nearby or sealant already applied, and returns a normal-looking number for the wrong feature. A sudden 5 mm jump in one sample is accepted as real; with the gain of 7 that becomes a 35 mm request (clamped to 30). Once the robot has moved towards the wrong edge, the next reading seems to confirm it: the wrong-feature lock.

Fix. Compare each reading with what is expected (Kalman filter gate), limit the jump allowed per sample, and use the Gocator's own quality output.

11 · Lost connections and missing data are not detected

What the code does. The return values of `initServer()` and `acceptConnection()` are not checked (`status_` is stored but never used). Send errors are printed and ignored. If no Gocator message arrives, the FOH simply holds its last target.

Why it is a problem. If the cable drops, the robot program stops, or the vision node crashes, the PC keeps running and the robot keeps dispensing with the last correction. Nobody is warned, so a whole bead can be laid with a stale correction.

Fix. A heartbeat in both directions. If no new reading arrives for N periods, or the robot's echo stops (item 9), hold the offset, stop dispensing and raise an alarm. Reconnect automatically.

12 · Mixed units and wrong fallback values

What the code does. `config.json` is in metres (setpoint −0.008, range ±0.0125). The fallback defaults in the constructor are a mix: `setpoint` 0.05, `max_rate_limit` 1.5, `max_accel_limit` 25, `sensor_range` ±12.5. The FOH comment says "new_target is in mm", and the start-up log prints the limit as "mm/s".

Why it is a problem. If a key is missing or misspelt in the config, the default is used silently. `max_rate_limit` would become 1.5 m/s, 150 times the intended 10 mm/s, and a range of ±12.5 *metres* would switch off the range check. During tuning the log shows

"Limit: 0.01 mm/s" when it is really 10 mm/s, which is confusing.

Fix. Use SI units (m, m/s, m/s²) everywhere, make every key required (stop with an error if one is missing), and correct the comments and log text.

13 · Logged data is lost on a crash

What the code does. Every 2 ms, `controlLoop` appends a row to a vector in memory. The file is only written in the destructor (`dump_data`).

Why it is a problem. That is 500 rows per second, about 1.8 million per hour, all held in RAM. If the node crashes or is killed, the destructor doesn't run and all data is lost, which is exactly when you need it. The rows also have no time stamp, only an index.

Fix. Write the log continuously from a background thread, flush regularly, and store a time stamp with each row.

14 · File paths fixed to one computer

What the code does. The config path is hard-coded as `/home/ws/Nov_27th/samu-dynamic-path-correction/…/config.json`; the logs go to `../../control_data_dump.csv`, relative to wherever the node is started.

Why it is a problem. On another PC, user or folder, the node stops at start-up with "Could not open config file". The logs land in a different place depending on where you launch from, and each run overwrites the previous one.

Fix. Pass the config path as a ROS parameter or launch argument, and write logs to a fixed log folder with a date and time in the file name.

15 · Old and mismatched files

What the code does. `pathcorrection_thread.cpp` and the Python node no longer match the current headers. The dummy publisher sends `Float64` on `/sensor/z_feedback` in mm, while the real node expects `Float32` on `/z_feedback` in metres. `USE_ZOH` points to a file that doesn't exist. There are unused globals and includes.

Why it is a problem. A new reader can't tell which files are the real ones. The dummy publisher can't drive the real node, so the PC logic can't be tested without the robot and Gocator.

Fix. Delete or archive the old files, and update the dummy to publish what the real node expects, so it can be used for offline tests.

16 · The Python prototype would stop the robot program

What the code does. `scripts/utils/ur_socket.py` sends `path_offset_set(…)` as text to port 30002.

Why it is a problem. Script text sent to port 30002 is run by the controller as a new program, which stops the program that is running, here the TP program with the nominal path. So that approach can't correct a path while it is being run. The C++ node avoids this because the URScript connects to the PC and reads the values itself.

Fix. Remove the prototype, or mark it clearly as not to be used.

17 · A NaN reading breaks the controller until restart

What the code does. No check for NaN ("not a number") anywhere.

Why it is a problem. Any comparison with NaN is false, so a NaN reading passes the range check. In the PID, the integral becomes NaN and stays NaN for ever; the clamp doesn't catch it (again, comparisons are false). The FOH target, speed and value all become NaN, and "nan" is sent to the robot every 2 ms until the node is restarted. NaN can come from the vision side when no edge is found, or from a division by zero.

Fix. Check `std::isfinite` on every input and before every send; treat NaN as an invalid reading (item 2 fix).

18 · FOH limits are offset limits; total TCP motion is not checked

What the code does. The FOH limits the Z offset to 10 mm/s and 1.2 m/s². The comment calls them "max TCP velocity".

Why it is a problem. Because the nominal path keeps Z constant, these are also the TCP's Z limits, so with Z only the effect today is small (for example $\sqrt{(100^2 + 10^2)} \approx 100.5$ mm/s total at 100 mm/s travel). It becomes important when lateral X and rotations are corrected too, because those add to the travel motion, and on tight curves where the nominal path already uses turning acceleration v^2/R. The values were also chosen by hand, not derived from the UR10e limits.

Fix. The per-step motion limiter in Part 2, which checks nominal + offset on all axes every 2 ms.

19 · No jerk limit

What the code does. At the start of each ramp the acceleration goes from 0 to the full 1.2 m/s² in one 2 ms step (tick 1 in the step-by-step table).

Why it is a problem. An instant change in acceleration is a small jolt. It is minor at these values, but it can excite vibration in the dispenser and is one more source of unevenness.

Fix. Add a jerk limit (how much the acceleration may change per step), or use a jerk-limited trajectory generator.

20 · Nothing checks the value just before it is sent

What the code does. `controlLoop` sends `-1 × foh_->update()` directly. The FOH itself has no clamp.

Why it is a problem. The FOH can overshoot its target slightly (about $v^2 / 2a = 0.01^2 / (2 \times 1.2) \approx 0.04$ mm). More importantly, any bug upstream (NaN, a wrong unit, a wrong sign) goes straight to the robot with no last line of defence.

Fix. A final guard just before sending: reject non-finite values, clamp to the max offset, and limit the change from the last sent value (Part 2, "Final guard").

21 · The FOH assumes every call is exactly 2 ms apart

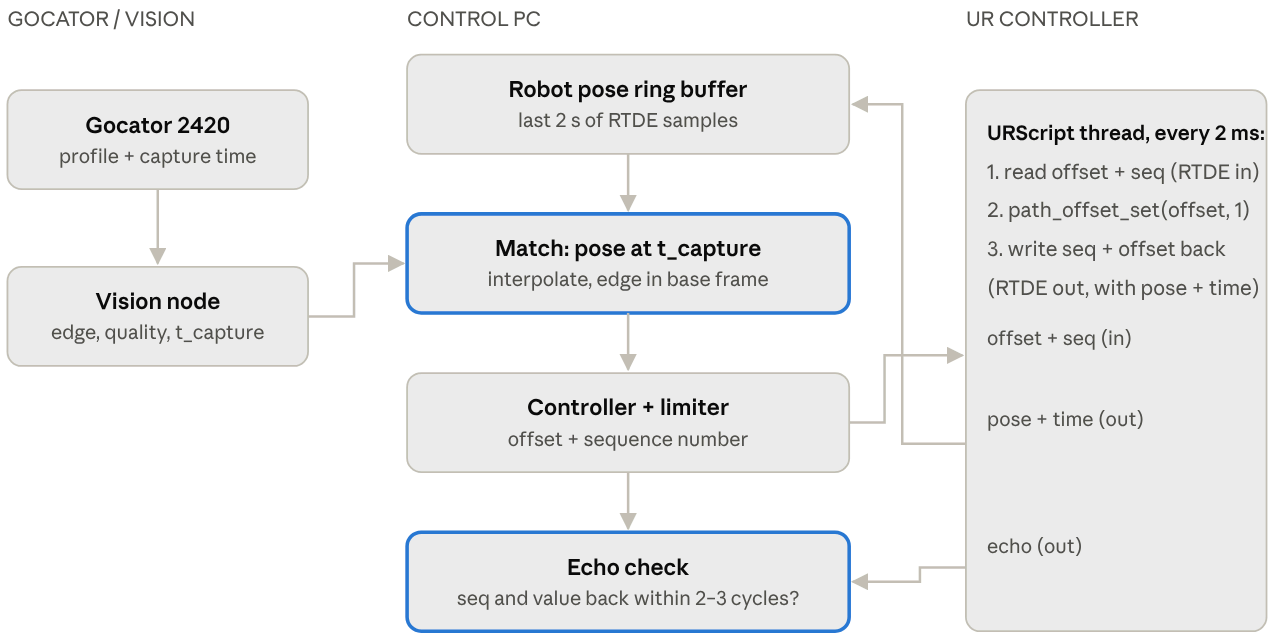
What the code does. `actuator_sample_time_` is fixed at 1 / 500 s; the real time between calls is never measured.

Why it is a problem. A ROS wall timer is not exact. If a call comes 3 ms after the previous one, the FOH still moves only one 2 ms step, so the real speed is lower than planned; if two calls bunch together, the robot gets two steps in one cycle, double speed for that moment.

Fix. Use the measured time between calls (clamped), or better, drive the loop from the robot's own 500 Hz cycle via RTDE.

How to add timestamps and robot feedback

This answers item 9. The idea: every reading carries the time it was taken, the PC always knows where the robot was at every moment, and the robot reports back every offset it applied.



Mismatch, missing echo or no new pose for N cycles → hold the offset, stop dispensing and raise an alarm.
RTDE in/out = UR's Real-Time Data Exchange interface (port 30004), synchronised with the robot's 500 Hz cycle.

Proposed: timestamped readings and read-back of the applied offset

1. **Time-stamp every Gocator profile.** The Gocator gives each frame a time stamp and frame number. The vision node should publish a stamped message (for example a custom message with `header.stamp` = capture time, the edge values and a quality flag) instead of a plain `Float32`. Convert the Gocator's clock to the PC clock once at start-up, or measure the fixed delay from capture to arrival and subtract it.
2. **Stream the robot state over RTDE.** RTDE (Real-Time Data Exchange, port 30004) sends robot data to the PC at 500 Hz: `timestamp`, `actual_TCP_pose`, `actual_TCP_speed`, `speed_scaling`, and output registers that the URScript writes. Libraries such as `ur_rtde` or the ROS 2 UR driver handle this. Keep the last few seconds in a ring buffer with the PC receive time.
3. **Match each reading to the robot pose.** For each profile, take the two RTDE samples just before and after its capture time and interpolate the pose. That gives the edge in the base frame, which is what the look-ahead buffer needs.
4. **Send the offset through RTDE input registers instead of the raw socket.** The PC writes the offset and an increasing sequence number into input registers; the URScript reads them every cycle. Registers always hold the latest value (no queue, item 8), use full precision (item 6) and are read in step with the robot cycle (item 21).

5. **Read back what the robot applied.** Right after `path_offset_set`, the URScript writes the same sequence number and offset into output registers (UR also has `path_offset_get(type)` for the offset currently applied). The PC checks that the echoed sequence number arrives within 2–3 cycles and the value matches what it sent. If not, the message was lost or the robot is stopped: hold, stop dispensing, alarm. You can also compare `actual_TCP_pose` with the nominal pose + offset to see how well the robot is following.
6. **Log everything with time stamps**: reading, capture time, robot pose, offset sent, offset echoed.

A minimal URScript thread for steps 4 and 5 (registers 24–47 are the ones meant for external RTDE clients):

```
thread offset_thread():
  while True:
    seq = read_input_integer_register(24)
    z   = read_input_float_register(24)
    path_offset_set([0, 0, z, 0, 0, 0], 1)
    write_output_integer_register(24, seq)
    write_output_float_register(24, z)
    sync()
  end
end
```

Part 2 · Improvements and future work

Everything below is proposed, not yet in the code. The quick fixes come first; they make the current PID + FOH safe and stable. The later sections replace it step by step with a look-ahead, limit-aware 6D design.

Improvements required

Fix stability and safety first; they need only small code changes. Then move to the architecture that removes the root causes: a look-ahead edge map with a filtered estimate, computed in base coordinates.

Step 1: quick fixes (days)

1. **Make the loop stable.** Use integral-only control with a per-sample gain below 1: `Kp = 0`, `Ki ≈ 0.5 × sensor_rate` (0.5 at 1 Hz, 2.5 at 5 Hz). Equivalent: `offset += k · error` with k = 0.3–0.7.
2. **Replace the fail-safe.** On an out-of-range or invalid reading: hold the last offset, count it, and stop dispensing after N consecutive invalid readings. Never command the full limit.
3. **Fix the start-up jump.** Reset the PID clock on the first sample, or use a fixed `dt = 1 / sensor_rate`.
4. **Raise the sensor rate** to what the Gocator actually delivers (20–100 Hz) and set `sensor_rate` to match.
5. **Send 6 decimals** and remove the per-cycle `std::cout`.
6. **Fix units and defaults** (all metres), check return values of `initServer` and `acceptConnection`, and make paths parameters.
7. **Verify the robot side:** message format vs `socket_read_ascii_float`, frame type (1 or 2), `path_offset_set_max_offset`, alpha filter and a watchdog.

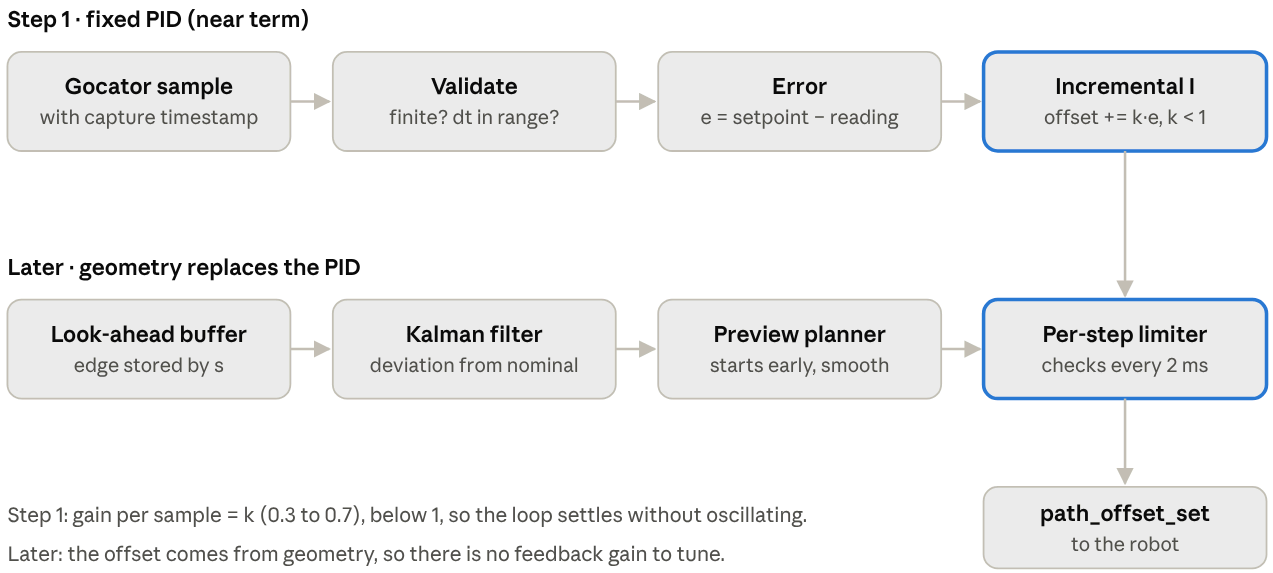
Step 2: robust interface (weeks)

1. **Use RTDE registers** instead of the raw socket: the robot always reads the latest value (no backlog), and the actual TCP pose and applied offset can be read back.
2. **Timestamp every profile** and match it to the robot pose at capture time.
3. **Add a sign-and-frame test** (+2 mm on one axis at a time) to commissioning.

Step 3: target architecture

1. **Look-ahead buffer:** convert each reading to base coordinates and store it by path position; use the value at the nozzle position, not the newest one.
2. **Kalman filter on the deviation from nominal** with a gate capped below the nearest wrong-feature distance, re-acquire after N rejections, coasting across cutouts.
3. **Geometric offset law:** `offset = inverse(nominal) ∘ edge frame ∘ setpoint`, then clamp, rate- and acceleration-limit (the current FOH logic can be reused here as the shaping stage).
4. **Extend axes** one at a time: Z, then X (lateral), then Rz, then Rx/Ry.
5. **Operating states:** pre-scan, tracking, coasting, re-acquire, fault stop, done.

PID-specific improvements



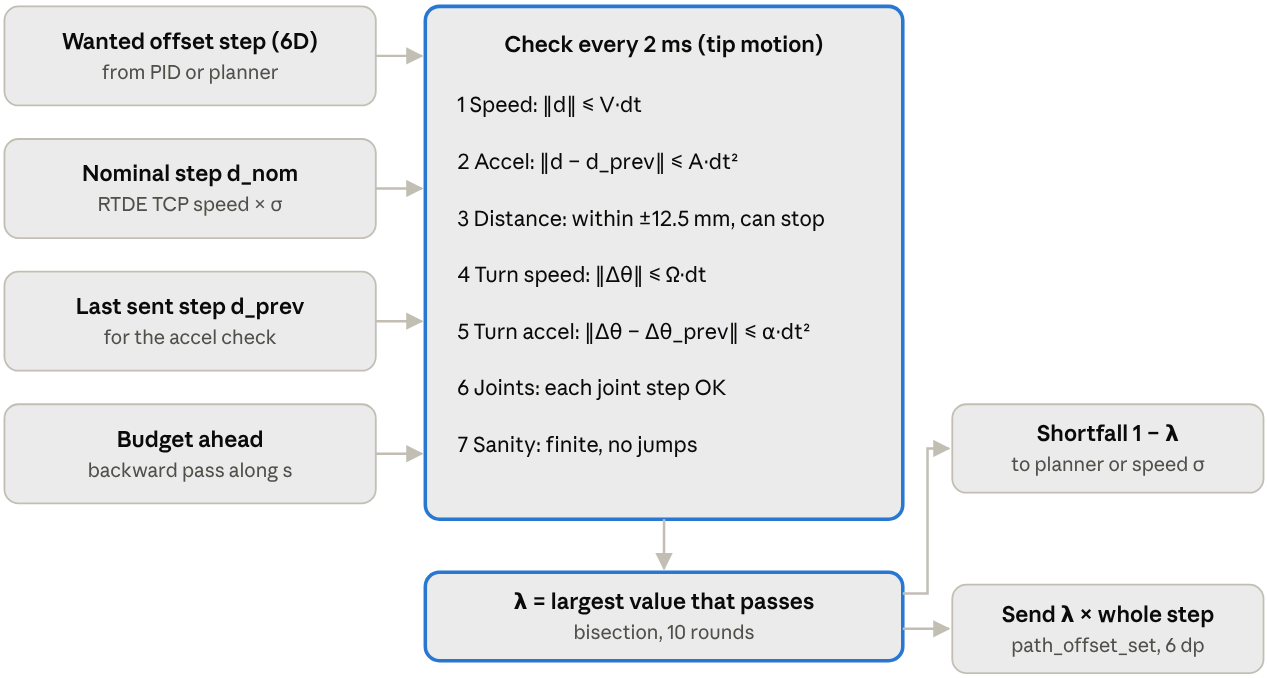
Future PID: fix it first, then replace it with the geometry pipeline

Near term (top row): fix the PID so its gain per sample is below 1. Later (bottom row): the look-ahead buffer, the Kalman filter and the preview planner compute the offset from geometry, and the PID is retired. Both routes feed the same per-step limiter.

1. **Make it incremental and integral-only.** `offset += k · error` with k = 0.3–0.7 per sample (equivalently Kp = 0, Ki = k · sensor_rate). This keeps the loop gain below 1 and removes the oscillation.
2. **Fixed or validated dt.** Use `dt = 1 / sensor_rate`, or measured dt clamped to [0.5, 2] × the nominal period; reset the clock on the first sample so start-up never winds up the integral.
3. **Reject non-finite inputs** (`std::isfinite`) before the PID, and never update the integral with NaN.
4. **Back-calculation anti-windup** (`integral += (u_sat − u) / (Ki · Tt)`) instead of undoing the last step; it recovers more smoothly after long saturation.
5. **Hold, don't jump, on bad data:** skip the update and keep the last output; count consecutive invalid samples and stop after N.
6. **Steady clock:** use `rclcpp::Clock(RCL_STEADY_TIME)` or `std::chrono::steady_clock` so system-time changes cannot produce odd dt values.
7. **Use timestamps from the message,** not the callback time, once Vision sends capture times.

8. **Unit tests** for step response, saturation, NaN and late samples (the class is header-only and easy to test).
9. **Later:** replace the PID on the raw reading with the Kalman filter on the deviation from nominal plus the look-ahead buffer; the offset then comes from geometry, and the PID is no longer needed.

FOH-specific improvements



d = nominal step + offset step, so the limits apply to the total tip motion, not just the offset.

Future FOH: the per-step motion limiter

The improved FOH becomes the per-step motion limiter. Every 2 ms it checks the total tip motion (nominal + offset) on all six axes and shrinks the whole step by λ when needed. The exact limits per tick are in the per-step limiter section of Part 2 and in its own spec.

1. **Derive the limits from the TCP budget,** not by hand (formulas below). Rename `max_rate_limit` / `max_accel_limit` to `offset_*` so the meaning is clear, and fix the comment.
2. **Add a jerk limit** (`max_jerk_limit`): limit how much the acceleration may change per step, for rounder ramp corners.
3. **Use the measured dt** (or a robot-synchronised clock) in the integration step, clamped to sane bounds.

4. **Add a final guard** after the FOH: reject non-finite values, clamp to ±maxOffset, and limit the change from the last sent value to `v_off_max · dt`.
5. **Plan the stop:** decelerate before reaching the target (`v ⩽ √(2 · a · remaining distance)`) so there is no overshoot near the limit.
6. **Send 6 decimals** so the robot receives the 0.02 mm steps the FOH computes.
7. **Per-axis and vector limits** when more axes are added: one limiter per axis plus a cap on the combined translation speed and a separate cap on rotation speed.
8. **Keep the robot-side guard:** `path_offset_set_max_offset` and the alpha filter in URScript.

Choosing the FOH limits from the TCP budget

The offset may only use what the nominal path leaves free:

```
v_{off,max} = m \cdot \sqrt{v_{TCP,max}^2 - v_{nom}^2} \qquad a_{off,max} = m \cdot (a_{TCP,max} - a_{nom})
```

How the two formulas are derived, step by step

Why a square root for speed but a simple minus for acceleration? The speed correction (X and Z) is always *across* the path, at a right angle to the travel direction, so the two combine like the sides of a right triangle (Pythagoras). The nominal acceleration can point in any direction: along the path when braking or speeding up, sideways in a curve. We can't count on a right angle, so we take the worst case, where the correction lines up with it and the two simply add. If a speed correction along the travel direction (Y) were ever used, it would also simply add: $v_{off,max} = m \cdot (V - v_{nom})$.

with $m \approx 0.8$ as a margin, v_{nom} the nominal path speed and a_{nom} its acceleration including v^2/R in curves. Use $v_{TCP,max} - v_{nom}$ when the offset direction is not perpendicular to travel. Per 2 ms step this gives `Δx_max = v_off,max · dt` and `Δv_max = a_off,max · dt`.

Parameter	Source
$v_{TCP,max}$, $a_{TCP,max}$	Stricter of the UR safety configuration and the sealing process limit; confirm with C219-free tests

Parameter	Source
v_nom, a_nom	Robot program (worst case) or measured live over RTDE
dt	Actual update period (2 ms)
Send resolution	Rounding step ÷ dt must stay below the speed budget (1 µm ÷ 2 ms = 0.5 mm/s)

A fixed budget (worst case of the program, in config) works with today's FOH code; a live budget (from RTDE each cycle) uses the full allowance on slow sections but needs robot feedback.

Final guard before sending (sketch)

```
double cmd = -1.0 * foh_->update();
if (!std::isfinite(cmd)) cmd = last_sent_;                          // never
send NaN or inf
cmd = std::clamp(cmd, minOffset_, maxOffset_);                      //
position limit
cmd = last_sent_ + std::clamp(cmd - last_sent_, -max_step_, max_step_);  //
per-update step limit
last_sent_ = cmd;
ur_socket_->sendOffset(cmd, "pose");                                // with 6
decimals
```

These are described in the companion document "Control Design: Dynamic Path Offset for Sealant Dispensing".

What to keep

- The two-rate structure (decide at sensor rate, actuate at 500 Hz).
- The FOH's velocity and acceleration limiting, as the final shaping stage.
- Config-driven parameters and the logging, written to disk continuously instead of only at shutdown.

Future work: target design

The recommended design measures the edge ahead, stores it by position along the path, and uses the known future reference to plan a smooth, limit-respecting offset trajectory that arrives at the nozzle on time. It replaces "PID on the current reading + FOH" with "estimate ahead + plan ahead", for all corrected axes (X, Z, Rz first; Rx, Ry later).

Target pipeline · 4 control stages

Stage A remembers the Gocator readings by path position (the look-ahead buffer). Stage B, the planner, uses that memory to move the offset early and within every limit. Stage C checks each 2 ms step once more before it reaches the robot. Stage D, the Kalman filter, cleans the readings and is the last option, added only if simple cleaning is not enough.

Read this first: the stages are one solution (look-ahead)

The stages below are **not** separate solutions. Together they are one idea, **look-ahead**, which has two halves and two helpers:

Stage	Plain name	What it does	Everyday comparison
A	**Look-ahead buffer**	*Remembers* each Gocator reading together with **where on the path** it was taken	Your eyes see a bump on the road ahead, and you remember where it is
B	**Planner**	Every 2 ms, *uses* that memory: "the nozzle is almost at that spot, so move now"	You lift your hand just as you reach the bump
C	Safety check (limiter)	Makes sure every 2 ms step is within what the robot can do	Never moving your hand faster than you safely can

Stage	Plain name	What it does	Everyday comparison
D	Cleaning with a Kalman filter (last option)	Throws out wrong readings and smooths the noise before they are remembered; added only if simple cleaning is not enough	Ignoring a shadow that only looks like a bump

The buffer alone does nothing; it only stores. The planner is what reads from it at the right moment. **Today's FOH does not disappear**: it moves inside the planner as its smoothing part. Today it smooths towards "the latest reading"; after the change it smooths towards "the remembered value for the spot just ahead of the nozzle".

In one line: **today = see → react immediately; look-ahead = see → remember where → act when the nozzle gets there.**

> The stages are one solution: look-ahead = remember (A) + use (B), with a safety…

How the code to-do list in Stage B fits. Steps 1–3 (time stamp, robot position, calculate and store at s + L) build **A**, the remembering half. Step 4 (every 2 ms take the value just ahead of the nozzle) is **B**, the using half. Step 5 (guard) is a simple first version of **C**. **D** (the Kalman filter) is left out of the first version; it is the last option (see the priority note below).

Priority: Kalman filter (Stage D): last option, not first. Current team view: the Kalman filter is not a priority now, because it is more complex to implement and tune, and it needs a model of how the system behaves. Note for clarity: the model needed here is small, not a full robot or system model. It is only the assumption that the edge is smooth along the path (height, slope and change of slope), plus two tuning values: the Gocator noise R and the allowed bending q. Even so, it needs recorded data and tuning time, so it is reasonable to start without it. **For the first version, use simple cleaning instead:**

- reject NaN and out-of-range readings;
- reject a reading that jumps more than gate_max from the previous stored value (this already removes cutouts and wrong edges);
- median of the last 3–5 readings, to remove single spikes;

- when reading the target from the buffer, average a few stored points *around* the target spot (before and after it, which is possible because the Gocator is ahead), to smooth the noise without adding lag.

Add the Kalman filter later, if these simple steps are not good enough (for example long cutouts where the edge must be predicted across a large gap, or very noisy readings).

Design goals

Goal	What it means	How the design meets it
Correct place	The correction for a spot is applied when the nozzle is at that spot	Position-indexed look-ahead map (Stage A)
No lag	The offset reaches its value when the nozzle arrives, also on curves and slopes	Preview planning: the reference is known L/v seconds ahead (Stage B)
Smooth	Continuous position, velocity and acceleration; bounded jerk	Jerk-limited trajectory generation (Stage B)
Within limits at every step	Offset distance, TCP velocity, TCP acceleration (and rotation rates) checked every 2 ms, per axis and combined	Constraint set and per-step guard (Stages B and C)
Robust	Noise, dropouts, cutouts and nearby features don't disturb the bead	Filtering, gating and smoothing (simple cleaning first; Kalman filter in Stage D as the last option)
Verifiable	What was planned, sent and applied is known	RTDE read-back and logging (Stage C)

Look-ahead (A and B together)

Look-ahead is one idea with two halves, kept together in this section:

- **A · Remember** (the look-ahead buffer): store each Gocator reading with *where on the path* it was taken.

- **B · Use** (the planner): every 2 ms, read the buffer at the spot just ahead of the nozzle and move the offset there smoothly. The planner is the **extension of the look-ahead buffer**: it is the part that turns the stored readings into the offset sent to the robot, and today's FOH code becomes its smoothing part.

The safety check (C) and, as the last option, cleaning with a Kalman filter (D) follow after this section.

A · Remember: the look-ahead buffer (Stage A)

This is the planned "use the Gocator history according to L and TCP speed", made robust.

1. **Path coordinate s.** Track the distance travelled along the nominal path (arc length), from RTDE poses or by integrating the actual TCP speed. Every stored point and every look-up uses s, not time, so speed changes (curves, start, stop, override) don't break it.
2. **Time synchronisation.** Each Gocator profile carries its capture time (hardware trigger or encoder preferred). The robot pose at that time is interpolated from 500 Hz RTDE samples on the same clock.
3. **Store edge geometry, not raw readings.** Each profile becomes an edge point and surface normal in base coordinates: `edge_base = robot_pose(t_capture) ∘ T_tcp_sensor ∘ edge_sensor`. The applied offset at capture time is already inside the robot pose, so stored points stay valid when the offset changes later.
4. **Match by pose, not by s + L.** On curves the Gocator is L ahead along the tool heading, not along the path. Each point is assigned the s of the nominal-path point it projects onto.
5. **Deviation from nominal.** Store `d(s) = edge(s) − nominal(s)` per axis (lateral, height, heading, tilts). The nominal path carries the big shape; d(s) is small and slow.
6. **Map window.** Keep points from slightly behind the nozzle to the newest sensed point; delete the rest.

A · Buffer design details

Idea in one line: the Gocator sees the edge L mm *before* the nozzle gets there. So we save every measurement with *where on the path* it was taken, and use it when the nozzle reaches that same place. We do not use "the latest value", and we do not use "the value from L/v seconds ago".

Why position, not time

- A time delay of L/v only works if v is constant. At corners and curves the TCP slows down (for example from 100 to 30 mm/s), so L/v changes all the time and a time-based buffer gives the wrong sample.
- Path position s (distance along the nominal path) does not care about speed. Sample taken at s = 250 mm is used when the nozzle is at s = 250 mm, whether the robot is fast or slow.

What each buffer entry stores

- `s` : path position of the *measured point* (not the robot), computed by projecting the point onto the nominal path.
- `p_base` : the measured edge point in the **base frame**: robot flange pose at the capture time (from RTDE, time-stamped) × Gocator mounting transform × Gocator point. Storing in base frame means the value stays correct even when the tool turns on a curve.
- `n_base` , `angle` : surface normal or edge angle, for the Rx/Ry/Rz terms when 6D is added.
- `t_capture` , `quality` (valid / rejected by the gate / dropout), and `d = deviation from the nominal path` at that s.

Inserting a sample (at the sensor rate)

1. Read the Gocator profile and its timestamp; pick the robot pose at that same timestamp (interpolate the RTDE log; do not use "now").
2. Transform the edge point to the base frame.
3. Project onto the nominal path to get s and the deviation d (lateral and vertical).
4. Run the Kalman gate on d (reject jumps at cutouts, re-acquire after N rejects).
5. Push into a ring buffer sorted by s. Drop entries more than one window behind the nozzle.

Reading at the nozzle (every 2 ms)

1. Current nozzle position on the path: s_noz = s(robot pose now) − small latency compensation (robot moves v·τ during the command delay τ).
2. Read the planner's target at s_noz + v·t_preview (preview distance), so the offset starts moving *before* the nozzle arrives. This is what removes lag.
3. Interpolate linearly between the two entries around that s (or use the smoothed KF/RTS value).

4. If there is no valid entry (dropout or cutout), hold the last valid deviation and fade the confidence; after a set distance with no data, ramp the offset to zero and raise a warning.

Window and memory

- The buffer only needs to cover the distance from the nozzle to the Gocator line: L_eff ≈ L plus a margin (the edge on a curve can be ahead or behind the laser line). With L = 60 mm and 1 mm spacing that is under 100 entries, so memory is not a concern.
- Spacing ds about 1 to 4 mm. At slow speeds you get many samples per mm; average them into one bin.

Curves and speed changes

- On curves, the laser line is not exactly L ahead *along the path*: it sits on a chord. Projecting the measured point onto the path (step 3 above) handles this.
- The nominal path is known, so the s of every point ahead is known; speed only changes how quickly we walk through the buffer.

Start and end of the bead

- Start: the nozzle starts at s = 0 but the buffer is empty for the first L mm. Options: a short pre-scan pass (move the Gocator over the first L mm before dispensing), or start dispensing only once the buffer is filled.
- End: the Gocator leaves the part first; keep the last L mm in the buffer and stop adding.

Pseudo-code

```
on_profile(meas, t):
    T_base_flange = interp_rtde_pose(t)
    p = T_base_flange * T_flange_gocator * meas.edge
    s, d = project_on_nominal(p)
    if kf.gate(s, d): buffer.insert(s, d, p, t)

control_step():                      # every 2 ms
    s_noz = project_on_nominal(nozzle_pose_now()).s
    target = buffer.interp(s_noz + v_now * t_preview)
    u_cmd  = planner.step(target)     # smooth, preview
    u_safe = limiter.step(u_cmd)      # per-step limits, see next section
    path_offset_set(u_safe, 1)        # base frame
```

B · Use: the planner, the second half of the look-ahead (Stage B, replaces the FOH ramp)

What Stage B does, in one sentence. Every 2 ms it decides the next offset value to send, using the clean edge from the buffer at the spot just ahead of the nozzle, and moves there smoothly within the robot's limits.

Why we need it even though we have the FOH. The FOH smooths the motion well. The problem is what it is fed and when. Today it receives the *latest* Gocator reading and ramps it in over the next sensor period, wherever the nozzle happens to be. But that reading belongs to a spot L = 60 mm ahead of the nozzle, and in one sensor period the robot travels v · T. So the correction lands in the right place only when v · T = L:

- at 100 mm/s it lands about **40 mm too late**;
- at 20 mm/s (slowing down for a corner) it lands about **40 mm too early**;
- only at exactly 60 mm/s is it in the right place, and that is a coincidence.

On top of that, with one reading per second the readings are v · T mm apart, and the FOH draws a straight line between them. Anything shorter, such as a small bump, is smoothed away or missed. The top picture shows both effects at 100 mm/s; the red areas are where the nozzle height is wrong.

Side view of a part with a 2 mm ramp and a short 1 mm bump · Gocator 60 mm ahea…

The fix: keep the smoothing, change what it is fed. The readings are stored by path position (the look-ahead buffer, Stage A). Every 2 ms the planner then does five steps:

The planner's five steps, repeated every 2 ms

1. **Where is the nozzle?** From the robot pose (RTDE), the nozzle is at path position s_n, for example 100 mm, moving at v = 100 mm/s.
2. **Look a little ahead.** The offset can't jump, and the whole system has a small delay (Gocator, PC, network, robot). So aim at the spot the nozzle will reach shortly: s_target = s_n + v · T_prev, with T_prev about 0.1 s (tuned on the robot). Here that is 110 mm. Read the target from the buffer: r(110 mm) = 1.0 mm.
3. **Move smoothly towards it.** The smoothing block computes the next small step from the current offset (0.50 mm) towards the target (1.0 mm), within the speed, acceleration and jerk limits; here +0.01 mm in this 2 ms. At first this can be today's FOH code; later a jerk-limited generator such as Ruckig.

4. **Check the step** with the per-step limiter (Stage C): nominal + offset must stay within the robot's limits, otherwise the step is shrunk.
5. **Send it** to the robot (RTDE). 2 ms later the nozzle is 0.2 mm further on, and the cycle repeats.

Because the target always comes from the right spot on the path, the correction lands in the right place **at every speed** (bottom picture). And because the buffer is filled 50–100 times per second instead of once, short bumps are caught.

What has to be done in the code (first version: FOH + look-ahead)

The first version keeps most of today's code. Highlighted boxes are new or changed.

What changes from today's code to the first look-ahead version

1. **Time-stamp every Gocator reading.** Vision publishes the reading together with its capture time, instead of a plain `Float32`. If that is not possible yet, the PC uses the arrival time minus a fixed delay measured once.
2. **Read the robot over RTDE at 500 Hz** (for example with the `ur_rtde` library): actual TCP position and speed. From it, keep a running **path position s_n** of the nozzle. Each cycle, add how far the nominal TCP moved (the actual TCP minus the offset we sent ourselves). No waypoint list is needed for this. Keep the last ~2 s of (time, s_n, offset sent) in a small history.
3. **Turn each reading into "the offset needed at that spot" and store it.** This replaces the PID. The Gocator moves with the robot, so its reading already includes the offset that was applied when it was taken. The offset needed at the spot the Gocator was looking at is therefore **needed = offset sent at capture time + (reading − setpoint)**. Store it in the buffer at path position **s_capture + L** (where the Gocator was looking). There is no feedback gain any more, so nothing can oscillate. Reject the reading if it is NaN, out of range, or jumps more than gate_max from the previous stored value.
4. **Every 2 ms, take the target from the buffer at s_n + v · T_prev** (interpolate between the two nearest stored points; hold the last value if there is none yet), and move the offset towards it with the FOH's speed and acceleration limits, plus braking so it stops on the target without overshooting.
5. **Guard and send.** Reject NaN, clamp to the max offset, limit the change per step, and send with 6 decimals.

File	Change
Vision node	Publish the reading with its capture time stamp (new message type)
New: RTDE reader (thread or node)	Actual TCP pose and speed at 500 Hz; path position s_n; history of (time, s_n, offset sent)
`pathcorrection.cpp`, `sensorCallback`	Replace the PID call with "needed offset" + gate + store in the buffer
`pathcorrection.cpp`, `controlLoop`	Target from the buffer at s_n + v·T_prev; limiter step; guard; remove the `× −1` (the sign is now in the formula)
`FirstOrderHold.hpp`	Use only its limit logic (speed and acceleration clamp), plus braking; no ramp over the sensor period
`socket_utils.cpp`	`setprecision(6)`
`config.json`	Add L, T_prev, gate_max; sensor rate 50–100 Hz

Code sketch (Z only, same units and sign convention as today's code, where a positive (reading − setpoint) gives a positive value sent; confirm the sign on the robot with a known step):

```
// --- RTDE reader, every 2 ms -------------------------------------
Vec3 p_tcp = rtde.getActualTCPPose();           // base frame, metres
Vec3 p_nom = p_tcp - Vec3(0, 0, last_sent_);    // remove our own offset
s_n_ += norm(p_nom - p_nom_prev_);  p_nom_prev_ = p_nom;
v_    = norm(rtde.getActualTCPSpeed());
history_.push({now(), s_n_, last_sent_});        // keep about 2 s

// --- Gocator callback, once per reading --------------------------
void sensorCallback(const ReadingStamped& m) {
  if (!std::isfinite(m.z) || m.z < range_min_ || m.z > range_max_) return;
  auto h = history_.at(m.stamp);                 // robot state at capture
  double needed = h.offset_sent + (m.z - setpoint_z_);
  if (!buffer_.empty() && std::abs(needed - buffer_.back().value) >
gate_max_) return;
  buffer_.push_back({h.s + L_, needed});         // spot the Gocator saw
  while (!buffer_.empty() && buffer_.front().s < s_n_ - 0.01)
```

```
buffer_.pop_front();
}

// --- control loop, every 2 ms ------------------------------------
void controlLoop() {
  double target = buffer_.interp(s_n_ + v_ * T_prev_, last_sent_);  // hold
if empty
  double err    = target - cur_;
  double v_des  = std::clamp(err / dt_, -v_max_, v_max_);
  double v_stop = std::sqrt(2.0 * a_max_ * std::abs(err));          // brake
in time
  v_des  = std::clamp(v_des, -v_stop, v_stop);
  vel_  += std::clamp(v_des - vel_, -a_max_ * dt_, a_max_ * dt_);
  cur_  += vel_ * dt_;
  double cmd = guard(cur_);                       // NaN, max offset, max step
  sendOffset(cmd);                                // 6 decimals
  last_sent_ = cmd;
}
```

After this works, the later steps are improvements on top, without changing the structure: a jerk-limited generator (Ruckig) in place of the limiter; the full per-step limiter of Stage C; the other axes (X, rotations) in the same way; and, only as the last option, the Kalman filter and smoother of Stage D between "needed offset" and the buffer.

Limits every step must respect. The planner aims to stay inside these; the per-step limiter in Stage C checks them again as a safety net.

Constraint	Per axis (X, Y, Z, Rx, Ry, Rz)	Combined TCP
Position	abs(p_off,i) ⩽ P_i (e.g. ±10 mm, ±5°); stay inside the Gocator range	—
Velocity	abs(v_off,i) ⩽ V_i	‖v_nom(s) + v_off‖ ⩽ v_TCP,max (vector length)
Acceleration	abs(a_off,i) ⩽ A_i	‖a_nom(s) + a_off‖ ⩽ a_TCP,max, with a_nom including v^2/R in curves
Jerk	abs(j_off,i) ⩽ J_i	—
Rotation	‖ω_off‖ ⩽ ω_max, ‖α_off‖ ⩽ α_max	Gocator / flange speed r · ω within limits (lever arm)

Constraint	Per axis (X, Y, Z, Rx, Ry, Rz)	Combined TCP
Joints (optional)	—	Joint speed and acceleration via the UR10e Jacobian, or monitored from RTDE

The nominal values v_nom(s) and a_nom(s) are known ahead from the robot program (or measured live), so the remaining budget is known for every point of the preview window.

Option B1 (start here): FOH-style smoothing, fed from the buffer

1. **First version: keep today's FOH code.** Only change its input: every 2 ms set its target to r(s_n + v · T_prev) from the buffer, instead of the latest PID output. This alone fixes the speed-dependent timing.
2. **Then replace the FOH with a jerk-limited generator** (for example the Ruckig library). Same idea, but it also limits jerk, rounds the corners of every move and never overshoots.
3. **The per-step limiter (Stage C) checks every step** for the combined nominal + offset motion, and shrinks it if needed.

Option B2 (later, only if C1 is not good enough): model predictive control (MPC). In plain words: instead of looking at one target point, MPC looks at the whole stretch of buffer ahead of the nozzle, works out the best offset motion for all axes together within all the limits, sends only the first step, and repeats every cycle. It handles big steps and tight curves best, but needs an optimisation solver and more tuning. Details:

- State per axis `[p, v, a]`, input = jerk, horizon = the preview window (e.g. 50–200 steps).
- Cost: weighted tracking error at the nozzle positions plus a small jerk penalty.
- Constraints: the full table above (vector norms as linearised polytopes).
- Solved as a quadratic program (e.g. OSQP, warm-started) at 100–500 Hz; the first step is applied, then the horizon moves on.
- MPC starts each move exactly as early as needed and shares the budget between axes optimally.

When the limits cannot be met (speed scaling). If a required change cannot be completed within the preview window under the limits (a large step, a tight curve), the planner reports it ahead of time and lowers the nominal speed through RTDE speed scaling. Lower speed frees TCP budget and lengthens the preview time; the speed returns afterwards.

Latency compensation. Measure the total delay τ (Gocator exposure and processing, Vision, network, RTDE, robot servo) and plan the value for time t + τ.

C · Safety check: per-step guard and robot interface (Stage C)

1. **Independent guard on the PC** after the planner, every 2 ms: reject non-finite values; clamp position; limit the change per step to `v_budget · dt` and the change of velocity to `a_budget · dt`; hold the last value on any violation and log it.
2. **RTDE, not a raw socket:** write the six offset values plus a counter into input registers; read back the actual TCP pose, TCP speed, applied offset (`path_offset_get`) and safety status from output registers.
3. **URScript loop** at 500 Hz: `path_offset_set(off, 2)` (TCP frame), `path_offset_set_max_offset(...)` as a hard limit, alpha filter near 1 (the command is already smooth), and a watchdog that holds or stops when the counter stops changing for 100 ms.
4. **Send full precision** (double registers, no 0.1 mm rounding).

Per-step limit check at every robot update (all axes)

Every 2 ms, before a value leaves the PC, the next offset step is checked against the **total** TCP motion, which needs the current TCP velocity and acceleration, not just the offset's. If any limit would be exceeded, the step is shortened; a value beyond a limit is never sent. This generalises today's FOH limiter (offset only, Z only) to the whole TCP and all axes.

Why the current TCP state is needed. The robot moves along the nominal path and the offset is added on top:

```
v_{TCP} = v_{nom} + v_{off} \qquad a_{TCP} = a_{nom} + a_{off}
```

The limits (UR safety, process, C219 path-offset checks) apply to the left-hand side. A 10 mm/s offset is harmless at 20 mm/s travel but can break a limit at 95 mm/s travel or in a tight curve, where a_nom already contains the turning acceleration v^2/R.

Inputs to the check

Input	Source	Note
Offset state p_off, v_off, a_off at step k	The guard's own record of what it last sent	Exact, no delay

Input	Source	Note
Nominal velocity and acceleration at step k+1	Robot program along the path coordinate s (predicted)	Known ahead; includes v^2/R in curves
Measured TCP velocity	RTDE `actual_TCP_speed` at 500 Hz	About one cycle late; used to monitor and correct the nominal estimate
Limits	Config: P per axis, V_TCP, A_TCP, J, ω, α, lever-arm speed	Stricter of robot safety settings and process limits, with margin
dt	Measured update period	Not assumed

Algorithm, every update k → k+1

1. Take the planner's candidate step for all axes: `Δp* = p_target(k+1) − p_off(k)`.
2. Derive what that step implies: `v_off* = Δp*/dt`, `a_off* = (v_off* − v_off(k))/dt`, `j* = (a_off* − a_off(k))/dt`.
3. Predict the total TCP motion: `v_TCP* = v_nom(k+1) + v_off*`, `a_TCP* = a_nom(k+1) + a_off*`.
4. Check every limit:
   - position per axis: abs(p_off(k) + Δp*) ⩽ P_i
   - distance per step: ‖Δp*‖ ⩽ v_off,max · dt
   - TCP velocity: ‖v_TCP*‖ ⩽ V_TCP,max
   - TCP acceleration: ‖a_TCP*‖ ⩽ A_TCP,max
   - jerk: abs(j*) ⩽ J per axis
   - rotation: ‖ω*‖ ⩽ ω_max, ‖α*‖ ⩽ α_max, and r · ‖ω*‖ within the speed limit at the Gocator and flange
   - stopping: the offset can still brake to rest before its position limit: $v_{off}^2 / (2 \cdot A_{available})$ ⩽ remaining distance
5. All satisfied → send the step. Otherwise find the largest fraction λ in [0, 1] of the step that satisfies all of them (closed form per limit, take the smallest, or about 10 bisection steps) and send `p_off(k) + λ · Δp*`. Log the limited step.
6. Update the guard's own state (p_off, v_off, a_off) from what was actually sent.
7. Monitor: compare the measured TCP speed with the predicted one; a persistent mismatch means a timing or model error and raises an alarm.

Worked example (illustrative limits): $V_{TCP,max}$ = 100 mm/s, $A_{TCP,max}$ = 1 m/s², travel 50 mm/s along Y, offset currently at rest, and the planner asks for a 0.2 mm Z step in one 2 ms update.

Check	Without limiting	Result
Offset velocity	0.2 mm / 2 ms = 100 mm/s	—
TCP velocity	$\sqrt{(50^2 + 100^2)} \approx 112$ mm/s	Exceeds 100 mm/s; velocity alone allows $\leq \sqrt{(100^2 - 50^2)} \approx 87$ mm/s
TCP acceleration	(100 mm/s − 0) / 2 ms = 50 m/s²	Far above 1 m/s²; allows a velocity change of only 1 m/s² × 2 ms = 2 mm/s
Step sent	λ limited by acceleration	2 mm/s × 2 ms = **0.004 mm** this update, then growing smoothly

Acceleration is usually the binding limit. That is why the planner (Stage B) must start moving early using the preview: the guard keeps every step safe, and the preview makes sure those safe steps still arrive on time.

Prediction versus measurement. Use the predicted nominal motion (from the program) plus the guard's own offset state for the check, because they have no delay. Use the measured RTDE values to monitor and to correct the prediction; avoid differentiating measured velocity to get acceleration without filtering, as it is noisy.

Code sketch (all axes)

```
Vec6 step = target_next - p_off;                        // candidate step, all
axes
double lam = 1.0;
lam = std::min(lam, posLimitFraction(p_off, step, P));  // position per axis
lam = std::min(lam, stepLimitFraction(step, vOffMax * dt));
for (int it = 0; it < 10; ++it) {                        // bisection on the
coupled TCP limits
  if (tcpVelOk(vNomNext, (lam * step) / dt) &&
      tcpAccOk(aNomNext, ((lam * step) / dt - v_off) / dt) &&
      jerkOk(((lam * step) / dt - v_off) / dt - a_off, dt) &&
      rotOk(lam * step, dt) && canStop(p_off + lam * step, (lam * step) /
dt)) break;
  lam *= 0.5;                                             // shrink until all
limits hold
}
Vec6 cmd = p_off + lam * step;
if (!allFinite(cmd)) cmd = p_off;                        // never send NaN
updateState(cmd, dt);                                    // p_off, v_off,
a_off
sendRTDE(cmd);                                           // full precision
```

D · Cleaning the Gocator readings with a Kalman filter + smoother (Stage D, last option)

The problem Stage D solves. The raw Gocator readings can't be sent to the robot directly:

- **Noise:** each reading jitters by about ±0.1 mm. Used directly, the nozzle would wobble.
- **Wrong readings:** at a cutout, hole, second edge or spatter, the Gocator returns a normal-looking number for the wrong feature (in the chart below, about −5 mm inside the cutout).
- **Missing readings:** sometimes no edge is found at all.

Stage D turns these raw readings into one clean, trustworthy edge line r(s) that the planner (Stage B) can follow. In the first version, the simple cleaning steps listed in "Read this first" do this job; the Kalman filter below is the last option, if they are not enough.

Why a Kalman filter? A Kalman filter is a "smart average". It uses one piece of knowledge about the part: **the edge is smooth**. Its height and position change gradually along the path; they don't jump 5 mm within 4 mm of travel. So for every new reading the filter can (1) predict where the edge should be from the readings before, (2) check whether the new reading agrees, and (3) blend the two. It also keeps track of how sure it is, which tells

us how far a reading may be from the prediction before it becomes suspicious. A plain moving average can do neither: it would average the −5 mm cutout readings into the edge, and it lags behind on slopes.

What it tracks. For each corrected axis (Z height, X lateral, later the angles), three numbers at path position s:

- **d**: how far the real edge is from the nominal path (mm)
- **d′**: the slope, how much d changes per mm of path (on a 2% ramp, d′ = 0.02)
- **d″**: the curvature, how fast the slope itself changes

It steps along the path position s (one step per reading, every 1–4 mm), not along time, so the travel speed doesn't matter.

What happens to each Gocator reading, and what the smoother adds at the nozzle

What happens for each new reading (example numbers for the Z axis):

1. **Predict.** From the last estimate: $d_{pred} = d + d' \cdot \Delta s + \tfrac{1}{2} \cdot d'' \cdot \Delta s^2$. Example: d = 1.00 mm, d′ = 0.02, Δs = 4 mm → d_pred = 1.08 mm. The filter also becomes a little less sure, by the process noise q (how much the edge is allowed to bend between two readings).
2. **Compare.** The difference ν = reading − d_pred (called the innovation). Example: reading 1.12 mm → ν = +0.04 mm. The filter also knows $\sqrt{S}$, the size of a *normal* difference (from its own uncertainty plus the Gocator noise R), for example 0.1 mm.
3. **Gate.** Accept the reading only if $|\nu| < \min(3 \cdot \sqrt{S}, \text{gate_max})$. Example: $3 \cdot \sqrt{S}$ = 0.3 mm, so ν = 0.04 is accepted. Inside the cutout the reading is −5 mm, ν ≈ −7 mm, so it is rejected. gate_max (for example 1 mm) is a hard cap set below the depth of the nearest wrong feature, so a wrong edge can never be accepted, even when the filter is unsure. NaN and out-of-range readings are rejected before this step.
4. **Update (accepted).** New estimate = d_pred + K·ν. K is a number between 0 and 1 that says how much to trust the reading compared with the prediction; the filter computes it itself from q and R. Noisy sensor → small K (follow the smooth line); precise sensor → larger K (follow the readings).
5. **Coast (rejected).** Keep the prediction: the edge is assumed to carry on smoothly across the gap. Count rejections in a row. After more than N in a row (N longer than the longest cutout, for example 15 readings = 60 mm), the edge may really have changed, so restart the filter from the next readings (re-acquire). If it goes on too long, stop dispensing and raise an alarm.

6. **Store** the estimate and its uncertainty in the look-ahead map at s.

What gate_max is. gate_max is a fixed safety cap, in mm, on how far a reading may be from the prediction and still be accepted. The gate is: accept if |reading − prediction| < min(3·√S, gate_max). It has two parts:

- **3·√S** is the filter's own idea of a normal difference, and it changes all the time. When the filter is confident it is small (for example 0.3 mm). When the filter is unsure, after a long cutout or right at the start, it grows (for example to 6 mm).
- **gate_max** is your fixed limit (for example 1 mm), chosen from the part's geometry. The gate always uses whichever of the two is smaller.

Why it is needed. Without the cap, an unsure filter would open its gate to 6 mm. A cutout reading 5 mm away would then be accepted, and the filter would lock onto the wrong feature. With gate_max = 1 mm the gate can never be wider than 1 mm, so the cutout is always rejected.

gate_max keeps the gate narrow even when the filter is unsure

How to choose gate_max. It must sit between two values taken from the part and the Gocator:

- **Larger than the biggest real change between two readings**, otherwise real edge changes are rejected. Example: steepest real slope × reading spacing + 3 × noise = 0.05 × 4 mm + 3 × 0.08 mm ≈ 0.45 mm.
- **Smaller than the distance to the nearest wrong feature**, otherwise wrong edges are accepted. Example: a cutout 5 mm deep, or a second edge 3 mm away.

In this example any value between about 0.5 mm and 3 mm works, and 1 mm is a sensible middle. Check both limits on real parts, especially the steepest real step or ramp: that is what a gate_max set too small would wrongly reject.

Why a smoother on top of the filter? The filter only knows readings taken *before* s, which causes two small errors (see the chart):

- at the corner of the ramp (s ≈ 140 mm) it lags behind for a few readings;
- across the cutout it carries on with the old slope and drifts, up to 0.5 mm too high at the end of the cutout.

But the Gocator is L mm ahead of the nozzle. By the time the nozzle reaches s, the readings from s up to s + L already exist. The **smoother** (a Rauch–Tung–Striebel smoother, run only over that window, hence "fixed-lag") goes back over the window and corrects each estimate using the readings *after* it as well. In the example it removes the corner lag and pulls the cutout back onto the true line using the first readings after the cutout. The maximum error drops from 0.5 mm to 0.1 mm, and the average from 0.11 mm to 0.04 mm. It adds **no delay**, because those later readings are taken anyway while the nozzle travels the distance L.

Simulated example: 2% ramp then a gentle bump, cutout from s = 180 to 212 mm, G…

What Stage D outputs. For every s between the nozzle and the Gocator, and for each corrected axis: the clean edge deviation r(s), its slope, and an uncertainty band (±). Stage B uses r(s) ahead of the nozzle to plan a smooth offset. A wide band (for example after a long coast) can be used to slow the robot or stop dispensing.

Tuning in practice.

- **R** (sensor noise): measure the Gocator on a flat, still part. For example σ = 0.08 mm gives R = 0.0064 mm².
- **q** (how much bending is allowed): too small and the estimate is stiff and cuts corners; too large and it follows the noise. Tune on recorded data from real parts.
- **gate_max**: below the distance to the nearest wrong feature (cutout depth, second edge).
- **N** (re-acquire): longer than the longest cutout or gap on the part.

When is "no lag" guaranteed?

The offset arrives on time when the time needed to make a change, within the limits, fits inside the preview time:

```
t_{move}(\Delta, V, A, J) \;\le\; \frac{L_{eff}}{v_{nom}} - \tau
```

For a rest-to-rest change Δ that does not reach the speed limit, `t_move ≈ 2V(Δ/A) + A/J`.

Example (illustrative limits): Δ = 2 mm height change, A = 1 m/s², J = 50 m/s³ gives t_move ≈ 0.09 + 0.02 = **0.11 s**. With L_eff = 50 mm, v_nom = 50 mm/s and τ = 50 ms, the preview is 1.0 − 0.05 = **0.95 s**: the planner starts about 0.11 s of travel (≈ 5.5 mm) before the change

and the nozzle meets it with zero lag. If the condition fails (a larger step, faster travel, shorter L), speed scaling slows the nominal path just enough to restore it.

The same check covers curves: heading and lateral changes are planned against the same preview, and a_nom includes the turning acceleration v^2/R.

Verification metrics

Metric	Target
Tracking error at the nozzle, per axis (RMS and max)	Within the process tolerance (e.g. ±0.3 mm Z, ±0.5 mm lateral; to confirm)
Lag between reference and applied offset (cross-correlation along s)	≈ 0 mm
Limit utilisation (velocity, acceleration, jerk, TCP budget)	Below 100 % with margin; every violation logged
Planned vs applied offset (from `path_offset_get` over RTDE)	Equal within the send resolution
C219 or protective stops	Zero
Speed-scaling events	Logged with the reason

Implementation roadmap

Phase	Content	Exit criterion
1	Timestamps, RTDE pose logging, path coordinate s; Stage A buffer with simple cleaning for Z (Kalman filter of Stage D only if needed), tested offline on logged runs	Estimated surface matches a slow reference scan
2	Stage B planner (option B1) + Stage C guard for Z on the robot (dispenser off, then on)	Zero lag and no C219 on slope and step coupons
3	Add lateral X and heading Rz with the combined TCP budget	Bead within tolerance on the tightest curve

Phase	Content	Exit criterion
4	MPC (C2) and speed scaling; latency calibration	Same accuracy at higher travel speed
5	Tilts Rx and Ry (perpendicularity)	Nozzle angle within tolerance on warped parts

The existing PID and FOH can stay as a fallback mode during phases 1–2; the FOH's velocity and acceleration limiter logic is reused inside the Stage C guard.

Future work: per-step motion limiter for variable TCP speed

The current FOH caps only the **offset** (10 mm/s, 1.2 m/s², Z only, fixed dt). It does not know how fast the robot itself is moving. But the robot's limits apply to the **total** motion: nominal path + offset. And the nominal speed is not the same everywhere: it drops at sharp corners, around curves and in the speed ramps.

The full design is in the separate doc *Design Spec: Per-Step Motion Limiter (FOH Replacement)* (https://claude.ai/artifact/37fVXRaVgeepFsNg1f1pTq). Summary:

1. Budget at each point of the path

For each path point s ahead, compute what is left for the offset after the nominal motion uses its share:

- Velocity: the offset is mostly sideways to the path, so budget $U(s) = \sqrt{(V_TCP^2 - v_nom(s)^2)}$.
- Acceleration: nominal accel is $a_nom = a_t \cdot \hat{t} + v^2 \cdot \kappa \cdot \hat{n}$ (speed-up/braking + turning). Budget $W(s) = A_TCP - \|a_nom(s)\|$.
- Apply a safety margin (for example 80% of the robot's real limits) and include joint-speed limits near singular poses.

What this shows (chart in the spec): on fast straights the velocity budget is smallest; in the braking and speed-up ramps around a corner the acceleration budget is smallest (only 0.2 m/s² left when the nominal ramp uses 0.8 m/s²). A single fixed FOH limit cannot fit both.

2. Backward pass (look ahead in the limits too)

If a tight spot is coming, the offset must already be slow enough to stop in time. Walking backward from the end of the window:

$U_{adm}(j) = \min(U(j), U_{adm}(j+1) + W(j)\cdot dt)$

This gives, at every point, the highest offset speed from which the robot can still slow down within the budget ahead.

3. Per-step check and λ-shortening

Every 2 ms the planner proposes the next offset step. The limiter checks:

- $\|v_{nom} + u\| \leqslant V_{TCP}$ and $\|a_{nom} + w\| \leqslant A_{TCP}$ (nominal + offset together, using the measured RTDE speed and speed-slider σ);
- offset speed ⩽ U_{adm} at the current point;
- distance: the step stays inside the max offset (±12.5 mm), with stopping margin $\sqrt{(2\cdot W\cdot d)}$ to the wall.

If any check fails, the step is shortened by a factor λ in [0, 1] (same direction, smaller size) instead of cutting one axis. NaN or missing values → hold the last safe offset.

The exact checks every 2 ms (500 Hz). At 500 Hz the robot takes a new target every dt = 2 ms, so every robot limit becomes a limit on *how far the tip may move in one tick* and *how much that move may differ from the previous tick's*. Here d_k is the total tip step this tick (nominal step + offset step) and d_{k-1} is last tick's total step. The limits are illustrative; take the real ones from the UR safety settings, with a margin.

#	Criterion	Checked each tick	Example limit per tick
1	TCP velocity (equals distance per tick)	$\|d_k\| \leqslant V\cdot dt$	V = 120 mm/s → 0.24 mm
2	TCP acceleration (change of step)	$\|d_k - d_{k-1}\| \leqslant A\cdot dt^2$	A = 1000 mm/s² → 0.004 mm
3a	Offset distance from nominal	each offset axis ⩽ max offset	±12.5 mm
3b	Stopping distance to that wall	offset speed² ⩽ 2·A_left·(distance to wall)	depends on position

#	Criterion	Checked each tick	Example limit per tick
4	Angular velocity (Rx, Ry, Rz)	$\|\Delta\theta_k\| \leqslant \Omega \cdot dt$	30 deg/s → 0.06°
5	Angular acceleration	$\|\Delta\theta_k - \Delta\theta_{k-1}\| \leqslant \alpha \cdot dt^2$	300 deg/s² → 0.0012°
6	Joint speeds (near singular poses)	each joint step ⩽ joint limit × dt	UR safety limits
7	Sanity	no NaN, no input jump	hold last safe step

The same limit leaves different room for the offset at different points, because the nominal step changes along the path:

Situation	Nominal step per tick	Nominal step change per tick	Room for offset step (sideways)	Room for offset step change
Fast straight, 100 mm/s	0.200 mm	0	$\sqrt{(0.24^2 - 0.20^2)}$ = 0.133 mm	0.004 mm
Braking ramp, 0.8 m/s²	about 0.120 mm	0.0032 mm	0.208 mm	only 0.0008 mm
Sharp corner R10, 30 mm/s	0.060 mm	0.00036 mm (turning)	0.232 mm	0.0036 mm

If the wanted step fails any check, the whole 6D offset step is shrunk by one factor λ (found by bisection, 10 rounds), so the step keeps its direction and just arrives a little later. If even λ = 0 fails, the nominal path alone breaks a limit: hold the offset and warn. The nominal step comes from the RTDE actual TCP speed × speed slider σ (fallback: the programmed TP speed).

4. Corners, ramps and curves

- Speed slider: velocity scales with σ and acceleration with σ², so budgets are recomputed from σ every step.
- Corners: the backward pass makes the offset settle *before* the braking ramp starts.

- Curves: the turning acceleration $v^2\kappa$ is included; a slower curve speed frees velocity budget.
- If the needed correction still does not fit, report the shortfall to the planner (start earlier) or request a lower σ. Dispenser flow follows actual TCP speed, so slowing down does not change bead size.

How it connects to the look-ahead buffer

Buffer (where is the edge at s?) → preview planner (smooth offset that arrives on time) → limiter (can the robot do this step?) → `path_offset_set`. The no-lag condition: the planned move time must fit in the preview distance, $t_{move} \leqslant L_{eff} / v - \tau$.

Future work: transforming the offset to the dispenser tip TCP

The Gocator measures in **its own frame**, mounted L mm ahead of the nozzle and possibly tilted. The robot applies the offset at the **process TCP** (the dispenser tip). The current code takes the Gocator Z value and sends it directly as a Z offset. That is only correct when the Gocator axis is parallel to the tool Z, the tool is not rotating, and the path is straight. For 6D correction the offset has to be transformed properly.

Transform chain, Gocator reading to dispenser-tip offset

Frames involved

Frame	What it is	Where it comes from
B (base)	robot base, fixed	robot
F (flange)	tool flange	RTDE actual pose, time-stamped
T (process TCP)	dispenser tip, Z along the nozzle	TCP calibration (`set_tcp` in the program)
G (Gocator)	laser line frame: x along the line, z = range	hand-eye calibration of the mount, T_FG

Step by step

1. **Gocator to base, at capture time.** p_B = T_BF(t_capture) · T_FG · p_G. Use the robot pose at the moment the profile was taken (interpolate the RTDE log), not the pose "now". Do the same for the surface normal n_B and edge direction.
2. **Store by path position s** in the look-ahead buffer (previous sections).
3. **Desired tip pose when the nozzle reaches s.** Position: p*(s) = edge point + lateral setpoint · (edge normal in the surface) + height setpoint · (surface normal). Orientation R*(s): tool Z aligned with the wanted angle to the surface normal (the process angle from the experts), tool X along the travel direction.
4. **Offset at the tip, in base frame.** Δp = p*(s) − p_nom(s), where p_nom(s) is the nominal TCP position from the TP path at the same s. Rotation: ΔR = R*(s) · $R_{nom}(s)^T$, sent as a rotation vector (axis × angle).
5. **Choose the frame for `path_offset_set`.** Type 1 (base): send Δp and the rotation vector as they are; this is the simplest because the buffer is already in base frame. Type 2 (tool): rotate into the current tool frame first, $\Delta p_T = R_BT^T \cdot \Delta p$, using the TCP orientation *now* (not at capture time). In both cases, keep only the axes you correct (for example ignore the travel direction Y).
6. **Limit, then send.** The per-step limiter works on the tip motion (nominal + offset at the TCP), so the transform must happen before the limiter.

```
on_profile(meas, t):                          # sensor rate
    T_BG = interp_rtde(t) * T_FG
    p_B, n_B = T_BG * meas.edge, R(T_BG) * meas.normal
    s = project_on_nominal(p_B)
    buffer.insert(s, p_B, n_B)

control_step():                               # every 2 ms
    s_tip = project_on_nominal(tcp_pose_now()).s
    p_star, R_star = desired_tip_pose(buffer.at(s_tip + preview), setpoints)
    dp = p_star - p_nom(s_tip)                # base frame
    dR = rotvec(R_star * R_nom(s_tip).T)
    if OFFSET_TYPE == 2: dp, dR = R_BT_now().T * dp, R_BT_now().T * dR
    path_offset_set(limiter.step(dp, dR), OFFSET_TYPE)
```

Things to watch

- **Rotation pivot.** A rotation offset should turn the tool *around the tip*, so the tip stays where it is. Check this on the robot: with the nozzle on a fixed mark, send a pure 2° Ry offset. If the tip moves, the rotation is pivoting somewhere else; add the compensating translation $\Delta p_c = (I - \Delta R) \cdot r$, where r is the vector from the pivot to the tip.
- **Lever arm to the Gocator.** When the tool rotates by $\Delta\theta$, the Gocator line (L ahead) swings by about $L \cdot \Delta\theta$: 60 mm × 1° ≈ 1 mm. The measurement itself is still correct because step 1 uses the pose at capture time, but this is why time-stamped poses are essential.
- **Inclined Gocator.** If the Gocator is tilted relative to the nozzle, its z (range) is not the tool Z. The rotation inside T_FG takes care of this; never use the raw Gocator z as a TCP Z.
- **Sign.** With the full transform the sign comes out of the maths (Δp = desired − nominal), so the `-1 *` in `controlLoop()` goes away.
- **Calibration.** Errors in T_FG or the TCP go straight into the bead position. Verify both with a known reference edge before trusting the correction.